

DAY 49



EXECUTION PLANS IN SQL SERVER

Understanding how queries actually run



THE REAL SCENARIO

Your query looks simple...

```
SELECT *
FROM Orders
WHERE CustomerId = 10;
```

You already:

- ✓ Added indexes
- ✓ Optimized code

Still slow on production data? 😞

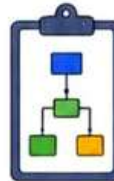
That's where
Execution Plans become important.

WHAT IS AN EXECUTION PLAN?

An execution plan shows how SQL Server executes a query step-by-step.

It helps identify:

- ✓ Table scans
- ✓ Index usage
- ✓ Joins
- ✓ Expensive operations



COMMON EXECUTION FLOW



WHY EXECUTION PLANS MATTER



- ✓ Find slow operations
- ✓ Detect missing indexes
- ✓ Improve query performance
- ✓ Understand SQL behavior

COMMON EXPENSIVE OPERATION

TABLE SCAN

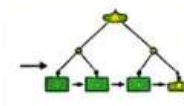
SQL Server checks the entire table.



Very slow on large tables 😞

INDEX SEEK

SQL Server jumps directly to matching rows using index.



Much faster! 🚀

MISTAKE I MADE

Looking only at query syntax 😞

Result:

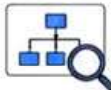
- ✗ Missed actual bottlenecks
- ✗ Slow production queries



REAL UNDERSTANDING

Two queries that LOOK similar can perform VERY differently.

Execution plan reveals what SQL Server is REALLY doing.



COMMON THINGS TO CHECK

- ✓ Table Scan vs Index Seek
- ✓ Expensive Joins
- ✓ Missing Indexes
- ✓ Sort Operations
- ✓ High-Cost Operators
- ✓ Implicit Conversions

IMPORTANT UNDERSTANDING

Query cost percentages are ESTIMATES, NOT absolute truth 🤖

Always combine:

- ✓ Execution plans
- ✓ Real query timing
- ✓ Actual workload analysis



ESTIMATED vs ACTUAL PLAN

ESTIMATED PLAN

SQL Server prediction



VS

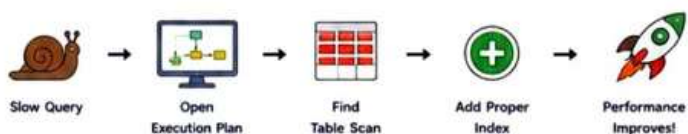
ACTUAL PLAN

What REALLY happened during execution



Actual plans are usually more useful.

EXAMPLE INVESTIGATION FLOW



★ Execution plans help you go from SLOW to FAST! 🚀

NORMALIZATION vs DENORMALIZATION (QUICK RECAP)

NORMALIZATION

- ✓ High consistency
- ✓ Less duplicate data
- ✓ Cleaner structure
- ✓ Better for writes

VS

DENORMALIZATION

- ✓ Faster reads
- ✓ Better reporting
- ✓ Duplicate data allowed
- ✓ Better for analytics

EXECUTION PLAN EXAMPLE (Actual Plan Snippet)



(Costs are relative estimates)

KEY INSIGHT

Good SQL tuning starts with understanding execution plans.

Not random optimization guesses! 😞

BEST PRACTICES

- ✓ Always check execution plan for slow queries
- ✓ Prefer actual execution plan
- ✓ Look for table scans
- ✓ Check missing index suggestions
- ✓ Review and test after changes

TOOLS TO VIEW PLANS

- SSMS - Include Actual Execution Plan (Ctrl + M)
- SSMS - Include Estimated Execution Plan (Ctrl + L)
- Azure Data Studio
- Query Store & DMVs



REMEMBER

- ★ The optimizer chooses the best plan (most of the time).
- ★ Good stats = Good plans
- ★ Data distribution impacts plans

TAKEAWAY

Execution Plans are one of the most powerful tools for performance tuning in SQL Server.



#dotnet

#sql

#backenddevelopment

#database

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds



DAY 50



SQL QUERY OPTIMIZATION TIPS

Practical ways to improve performance



THE REAL SCENARIO

Your API works perfectly in development...

But in production:

- ❌ Queries become slow
- ❌ CPU usage increases
- ❌ Reports take forever 😞



Writing SQL that **WORKS** is different from writing SQL that **SCALES**.

WHAT IS QUERY OPTIMIZATION?

Improving SQL queries so they:

- ✅ Execute faster
- ✅ Use fewer resources
- ✅ Scale better with large data



★ Good queries = Better performance, Scalability & Lower infrastructure cost

COMMON PERFORMANCE KILLERS

- ❌ Missing indexes
- ❌ SELECT *
- ❌ Large joins
- ❌ Long transactions
- ❌ Poor filtering
- ❌ Non-sargable queries



Identify bottlenecks first, then optimize **smartly**.

1 AVOID SELECT *

❌ Bad

```
SELECT *
FROM Orders;
```



Many Columns

✅ Better

```
SELECT OrderId, TotalAmount
FROM Orders;
```



Only Needed Columns

✅ Fetch only required columns.

2 ADD PROPER INDEXES

❌ Without Indexes

Table scans happen



✅ With Indexes

Faster lookups



```
CREATE INDEX IX_Orders_CustomerId
ON Orders(CustomerId);
```

✅ Indexes are one of the biggest performance boosters.

3 AVOID FUNCTIONS IN WHERE

❌ Bad

```
WHERE YEAR(OrderDate) = 2025
```



Prevents index usage sometimes.

✅ Better

```
WHERE OrderDate >= '2025-01-01'
AND OrderDate < '2026-01-01'
```



✅ Write sargable (search-argument-able) conditions for better index usage.

4 USE EXISTS INSTEAD OF COUNT (SOMETIMES)

❌ Bad (For existence check)

```
SELECT COUNT(*)
FROM Orders
WHERE CustomerId = 10;
```



✅ Better

```
IF EXISTS (
  SELECT 1 FROM Orders
  WHERE CustomerId = 10
)
```



✅ EXISTS stops at first match. Faster!

5 AVOID UNNECESSARY JOINS

Too many joins:

- ❌ Increase complexity
- ❌ Slow queries



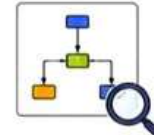
Always ask yourself: "Do I really need this table?"

✅ Only join what you truly need.

6 ANALYZE EXECUTION PLANS

Execution plans help identify:

- ✅ Table scans
- ✅ Missing indexes
- ✅ Expensive operators



✅ Your best friend for performance debugging.

MISTAKE I MADE

Optimizing queries blindly 😞

Result:

- ❌ No real improvement
- ❌ Added unnecessary complexity



REAL UNDERSTANDING

Optimization should be:

- ✅ Measured
- ✅ Tested
- ✅ Evidence-based



Not guesswork.

IMPORTANT UNDERSTANDING

Premature optimization can waste time.

First:

1 Identify bottlenecks

Then:

2 Optimize carefully



REAL-WORLD OPTIMIZATION FLOW



KEY INSIGHT

Good optimization improves **BOTH**:

- ✅ User experience
- ✅ Infrastructure cost



TAKEAWAY

SQL optimization is about writing scalable and efficient queries. Always optimize based on real bottlenecks, not assumptions.



SCENARIO

If your production database becomes slow, would you optimize based on assumptions or actual measurements?



COMING UP NEXT (DAY 51)
Entity Framework Core Basics
- how .NET applications interact with databases



BEST PRACTICES

- ✅ Write efficient queries
- ✅ Use indexes wisely
- ✅ Analyze execution plans
- ✅ Test with real data
- ✅ Monitor and improve regularly

TOOLS TO HELP

- Execution Plans (SSMS)
- Index Usage Stats
- Query Store
- SQL Profiler / Extended Events
- DMVs (sys.dm_exec_*)

REMEMBER

- ★ Understand first
- ★ Measure always
- ★ Optimize what matters
- ★ Re-test & validate

#dotnet #sql #Backenddevelopment

#database #100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds



DAY 51



ENTITY FRAMEWORK CORE BASICS

How .NET applications interact with databases



THE REAL SCENARIO

Your application needs to:

- ✓ Save users
- ✓ Fetch orders
- ✓ Update products
- ✓ Delete records

Should we write raw SQL for EVERYTHING? 😞



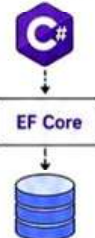
That's where Entity Framework Core comes in.



WHAT IS ENTITY FRAMEWORK CORE?

EF Core is an ORM (Object Relational Mapper).

It helps .NET applications interact with databases using C# objects instead of writing SQL everywhere.



WHAT EF CORE PROVIDES

- ✓ LINQ queries
- ✓ Change tracking
- ✓ Migrations
- ✓ Relationships handling
- ✓ CRUD operations
- ✓ Async database operations



TRADITIONAL SQL APPROACH

```
SELECT *
FROM Users
WHERE Id = 1;
```



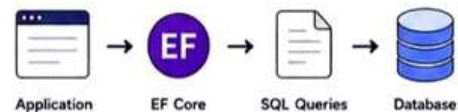
EF CORE APPROACH

```
var user = await _context.Users
    .FirstOrDefaultAsync
    (x => x.Id == 1);
```



- ✓ Cleaner integration with .NET code.

TYPICAL FLOW



DBCONTEXT EXAMPLE

```
public class AppDbContext : DbContext
{
    public DbSet<User> Users { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<Product> Products { get; set; }
}
```

- 1 DbContext manages database interaction.



ADDING DATA

```
var user = new User
{
    Name = "Srinija",
    Email = "srinija@example.com"
};
_context.Users.Add(user);
await _context.SaveChangesAsync();
```

- ✓ Changes are tracked and saved to database.



MISTAKE I MADE

Trusting EF Core blindly 😞

- Result:
- ✗ Slow generated queries
 - ✗ N+1 query problems
 - ✗ Performance issues



Always check what SQL is being generated under the hood.



REAL UNDERSTANDING

EF Core is VERY powerful BUT...

Backend developers should still understand:

- ✓ SQL
- ✓ Indexing
- ✓ Execution plans
- ✓ Query performance

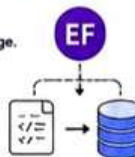


IMPORTANT UNDERSTANDING

EF Core does NOT remove the need for database knowledge.

It only:

- ✓ Simplifies interaction.



WHY COMPANIES USE EF CORE

- ✓ Faster development
- ✓ Cleaner code
- ✓ Strong .NET integration
- ✓ Productivity improvement



COMMON FEATURES



LINQ



Migrations



Relationships



Lazy / Eager Loading



Transactions

KEY INSIGHT

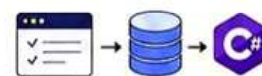


ORMs improve productivity BUT Performance awareness still matters.



TAKEAWAY

- ✓ EF Core helps .NET applications interact with databases using C# objects.
- ✓ Understanding both EF Core AND SQL is important for backend developers.



NEXT UP (DAY 52)

LINQ in .NET
Querying data using C#



SCENARIO FOR YOU

If your ORM generates inefficient SQL queries, would knowing only C# be enough to solve the problem?



REMEMBER

Great applications need great code AND great database interactions! 🙌

#dotnet

#efcore

#Backenddevelopment

#sqlserver

#100DaysOfCode

#LearningInPublic

#BackendJourney

#SrinijaBuilds





LINQ in .NET

Querying data using C#

.NET



THE REAL SCENARIO

Your application needs to:

- ✓ Filter users
- ✓ Sort products
- ✓ Search orders
- ✓ Group data

Should we write loops everywhere manually? 😞



That's where LINQ comes in.

WHAT IS LINQ?

LINQ = Language Integrated Query

It allows developers to:

- ✓ Query collections
- ✓ Filter data
- ✓ Sort data
- ✓ Transform data

using C# syntax itself.



COMMON LINQ OPERATIONS

- 🔍 Where → Filtering
- 📦 Select → Projection
- ↕ OrderBy → Sorting
- 👥 GroupBy → Grouping
- ❓ Any → Existence check
- 🔍 FirstOrDefault → First match

TRADITIONAL LOOP APPROACH

```
var activeUsers = new List<User>();
foreach(var user in users)
{
    if(user.IsActive)
    {
        activeUsers.Add(user);
    }
}
```

VS

LINQ APPROACH

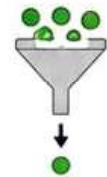
```
var activeUsers = users
    .Where(x => x.IsActive)
    .ToList();
```



Cleaner & more readable 🙌

FILTERING EXAMPLE

```
var users = _context.Users
    .Where(x => x.IsActive);
```



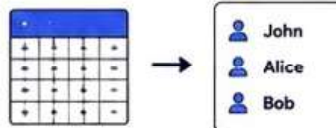
SORTING EXAMPLE

```
var orders = _context.Orders
    .OrderByDescending(x => x.CreatedDate);
```



PROJECTION EXAMPLE

```
var names = users
    .Select(x => x.Name);
```



MISTAKE I MADE

Writing overly complex LINQ queries 😞

Result:

- ✗ Hard readability
- ✗ Difficult debugging
- ✗ Poor generated SQL sometimes



LINQ is powerful, BUT simpler queries are usually easier to maintain.



IMPORTANT UNDERSTANDING

LINQ queries against EF Core do NOT run immediately.

They often use: Deferred Execution

Query executes when methods like

ToList() FirstOrDefault() Count()

are called.



LINQ TO OBJECTS vs LINQ TO ENTITIES

LINQ to Objects

Works on in-memory collections.



LINQ to Entities

Converts LINQ into SQL queries.



★ Very important difference!



WHY COMPANIES USE LINQ

- ✓ Cleaner code
- ✓ Better readability
- ✓ Strong .NET integration
- ✓ Faster development



KEY INSIGHT

LINQ improves developer productivity BUT developers should still understand what SQL gets generated underneath.

LINQ Query

EF Core

SQL



TAKEAWAY

- ✓ LINQ helps query and manipulate data using C# syntax.
- ✓ Clean LINQ queries improve readability and maintainability.



NEXT UP (DAY 53)

Lazy Loading vs Eager Loading in EF Core – understanding data loading strategies



SCENARIO FOR YOU

If your LINQ query generates inefficient SQL underneath, how would you identify the problem?



REMEMBER

Good LINQ + Good SQL understanding = Scalable & High performance applications! 🙌





Lazy Loading vs Eager Loading in EF Core

Understanding data loading strategies

.NET

Entity Framework



THE REAL SCENARIO

You fetch an Order from the database...

Later, you access:

- Order.Customer
- Order.Items

Suddenly:

- Multiple unexpected SQL queries execute 😞

That's where loading strategies in EF Core become important.

WHAT IS LAZY LOADING?

Related data loads ONLY when accessed.

```
var order = _context.Orders.First();

var customer = order.Customer;
```

Accessing Customer may trigger another SQL query.



LAZY LOADING BEHAVIOR



PROS

- Simpler initial queries
- Loads data only when needed

PROBLEMS

- Hidden database queries
- N+1 query problems
- Performance issues in loops

WHAT IS EAGER LOADING?

Related data loads immediately using Include().

```
var orders = _context.Orders
    .Include(x => x.Customer)
    .ToList();
```

EF Core fetches everything together.



EAGER LOADING BEHAVIOR



PROS

- Fewer database round trips
- Better performance in many cases
- More predictable queries

PROBLEMS

- Fetching unnecessary data sometimes
- Large joins can become expensive



MISTAKE I MADE

Using Lazy Loading everywhere 😞

Result:

- Too many hidden queries
- Slow APIs in production



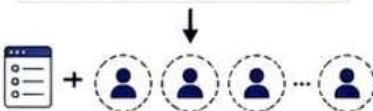
There is NO "always best" strategy.

Choose based on:

- Data size
- Usage pattern
- Performance needs

N+1 QUERY ISSUE

1 query for orders
+ 100 queries for customers 😞



Very common with Lazy Loading.

REAL-WORLD RECOMMENDATION

Many teams prefer:

- Explicit loading
- Eager loading carefully



Because hidden queries become easier to control.



IMPORTANT UNDERSTANDING

Always monitor:

- Generated SQL
- Query count
- Database round trips

Not just C# code 🐞



LAZY LOADING vs EAGER LOADING

Aspect	Lazy Loading	Eager Loading
When data loads?	When accessed	Upfront with Include()
Queries	Multiple queries (may be many)	Single query (with joins)
Performance	Can be slow (N+1 issues)	Generally better, more predictable
Data fetched	Only what you access	Everything requested upfront
Best for	Small/rarely used navigation data	APIs, reporting, list pages
Risk	Hidden queries, hard to detect	Over-fetching unnecessary data

KEY INSIGHT

ORM convenience can sometimes hide expensive database behavior.



TAKEAWAY

- Lazy Loading loads data when accessed.
- Eager Loading loads related data upfront using Include().
- Understanding query behavior is critical for performance.



NEXT UP (DAY 54)

Tracking vs No Tracking in EF Core - improving query performance



SCENARIO FOR YOU

If your API suddenly executes hundreds of unexpected queries, how would you investigate whether lazy loading is causing it?



REMEMBER

- Choose the right loading strategy.
- Monitor your queries.
- Performance starts with awareness!





in EF Core

Improving Query Performance



THE REAL SCENARIO

Your API only needs to:

- ✓ Display data
- ✓ No updates
- ✓ No modifications
- ✓ Just reading records



But your query is still using extra memory & resources! 😞



WHAT IS TRACKING?

By default, EF Core tracks the entities it loads.

- ✓ EF Core remembers changes
- ✓ Detects modifications
- ✓ Updates DB on SaveChanges()



WHAT IS NO TRACKING?

EF Core skips change tracking.

Use `AsNoTracking()`

```
var users = await _context.Users
    .AsNoTracking()
    .ToListAsync();
```



TRACKING EXAMPLE

```
var user = await _context.Users
    .FirstOrDefaultAsync(x => x.Id == 1);
user.Name = "Srinija";
await _context.SaveChangesAsync();
```

✓ EF Core knows what changed.

TRACKING FLOW



BENEFITS OF NO TRACKING

- ✓ Faster queries
- ✓ Less memory usage
- ✓ Better read performance
- ✓ Ideal for reports & APIs



IMPORTANT UNDERSTANDING

With No Tracking:

```
user.Name = "Updated";
```



`SaveChanges()` **won't** automatically update it.

TYPICAL USE CASES

Use **TRACKING** When:

- ✓ Updating data
- ✓ Editing records
- ✓ Business workflows

Use **NO TRACKING** When:

- ✓ Read-only APIs
- ✓ Dashboards
- ✓ Reports
- ✓ Large result sets



MISTAKE I MADE 😞

- ✗ Using Tracking for every query
- ✗ Unnecessary memory usage
- ✗ Slower performance
- ✗ Extra overhead



TRACKING vs NO TRACKING COMPARISON

Aspect	Tracking (Default)	No Tracking (AsNoTracking)
Change Detection	Yes	No
Memory Usage	Higher	Lower
Query Performance	Slower (extra overhead)	Faster
Save Changes Support	Yes	No
Best For	Updates, Edits, Workflows	Read-only, Reports, APIs
Overhead	High	Low

COMMON RULE



WHY COMPANIES USE NO TRACKING

- ✓ Faster API responses
- ✓ Reduced memory consumption
- ✓ Better scalability



KEY INSIGHT

Small EF Core choices can have a BIG impact on performance.



PERFORMANCE IMPACT

With Tracking

- ✗ More Memory
- ✗ Slower Queries



With No Tracking

- ✓ Less Memory
- ✓ Faster Queries



TAKEAWAY

- ✓ Tracking helps EF Core detect changes for updates
- ✓ `AsNoTracking()` improves performance for read-only queries
- ✓ Use the right approach based on your use case
- ✓ Small decisions = Big performance impact 🚀



NEXT UP (DAY 55)

EF Core Migrations
Managing database
schema changes safely



SCENARIO FOR YOU 🤔

If an API only reads data and never updates it, would you still pay the cost of tracking every entity? ❓



REMEMBER

Choose the right approach.
Performance will always thank you! 🙌





EF Core MIGRATIONS

Managing database schema changes safely

.NET



THE REAL SCENARIO

Your application is already running in production.

Now you need to:

- ✓ Add a new column
- ✓ Create a new table
- ✓ Modify relationships

Should you manually update every database?



That's where EF Core Migrations come in.

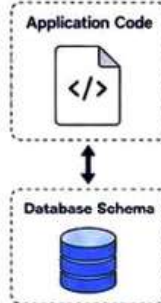
WHAT ARE EF CORE MIGRATIONS?

Migrations are version-controlled database schema changes.

They help keep:

- ✓ Application code
- ✓ Database structure

in sync.



EXAMPLE

Suppose we add a new property:

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

Previously ✗

No Email column

Id	Name
1	Srinija
2	Alex

Now ✓

Email column required

Id	Name	Email
1	Srinija	s@mail.com
2	Alex	a@mail.com

MIGRATION FLOW



Update Model Create Migration Review Changes Update Database

CREATE MIGRATION

Add-Migration AddUserEmail

EF Core generates migration files automatically.

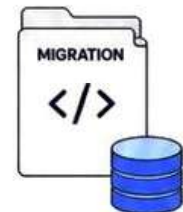
APPLY MIGRATION

Update-Database

Database schema gets updated. ✓

WHAT MIGRATIONS HANDLE

- ✓ New tables
- ✓ New columns
- ✓ Foreign keys
- ✓ Indexes
- ✓ Relationships



MISTAKE I MADE

Running migrations without reviewing generated SQL 😞



Result:

- ✗ Unexpected schema changes
- ✗ Risky production deployments
- ✗ Difficult rollbacks

REAL UNDERSTANDING

Migrations are powerful, BUT...

Never blindly apply them in production.

- ✓ Review changes
- ✓ Test in lower environments
- ✓ Take backups when needed



WHY COMPANIES USE MIGRATIONS

- ✓ Version control for database changes
- ✓ Team collaboration
- ✓ Repeatable deployments
- ✓ Easier environment management



COMMON WORKFLOW



Developer Changes Model Generate Migration Commit to Source Control Deploy Migration Database Updated

BEST PRACTICES

- ✓ Use meaningful migration names
- ✓ Review generated code
- ✓ Test before production deployment
- ✓ Keep migrations small and focused
- ✓ Don't edit old migrations after deployment



INTERVIEW INSIGHT

What is the difference between:

Add-Migration



Generates migration files.

VS

Update-Database



Applies them to the database.

IMPORTANT UNDERSTANDING

A migration is not just code.
It is a database change.

Database changes can affect:

- Data integrity
- Performance
- Availability



KEY INSIGHT

Good schema management is just as important as good application code.



TAKEAWAY

- ✓ EF Core Migrations help manage database schema changes safely and consistently.
- ✓ Always review and test migrations before deploying them.
- ✓ Use migrations as part of your disciplined development workflow.



NEXT UP (DAY 56)

Repository Pattern in .NET - benefits, drawbacks, and real-world usage



SCENARIO FOR YOU

If your production database contains millions of records, would you apply a migration without reviewing the generated changes first?



REMEMBER

- Review
- Test
- Backup
- Deploy with confidence! 🚀





Repository Pattern in .NET

Benefits, drawbacks, and real-world usage →



1. REAL SCENARIO

Your application needs to:

- ✓ Fetch users
- ✓ Save orders
- ✓ Query products
- ✓ Access multiple data sources



Should your business logic directly interact with Entity Framework everywhere?

That's where the Repository Pattern comes in.



2. WHAT IS REPOSITORY PATTERN?

The Repository Pattern acts as a layer between business logic and data access.



Its goal is to:

- ✓ Separate data access logic
- ✓ Improve maintainability
- ✓ Increase testability

</> 3. EXAMPLE

Without Repository (Direct EF Core)

```
var user = _context.Users
    .FirstOrDefault(x => x.Id == id);
```

With Repository

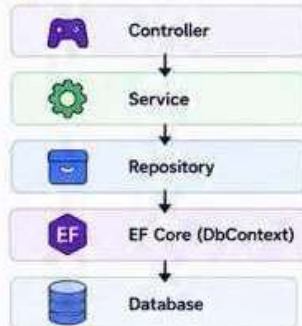
```
var user = _userRepository
    .GetById(id);
```



Business logic depends on the repository abstraction, not on EF Core directly.



4. TYPICAL FLOW



5. WHY TEAMS USE REPOSITORIES

- ✓ Cleaner architecture
- ✓ Better separation of concerns
- ✓ Easier unit testing
- ✓ Centralized data access logic
- ✓ Easier to change or mock data sources



6. COMMON PITFALL

Creating generic repositories for EVERYTHING.

```
Add()
Delete()
Update()
GetAll()
...
```



for every entity.

- ✗ Extra abstraction
- ✗ Duplicate EF Core functionality
- ✗ Harder maintenance



7. PRODUCTION INSIGHT



Many modern .NET teams debate whether repositories are needed when using EF Core. Why? Because DbContext already behaves like a Repository + Unit of Work.



In some projects: Repository Pattern adds value.



In others: Direct EF Core usage is simpler.



The right choice depends on project complexity, team preference, and long-term goals.



8. PROS vs CONS

PROS

- ✓ Better testability
- ✓ Cleaner separation of concerns
- ✓ Easier swapping of data sources
- ✓ Supports SOLID principles

VS

CONS

- ✗ More code and boilerplate
- ✗ Additional abstraction layer
- ✗ Can duplicate EF Core capabilities
- ✗ Overuse can make the codebase complex



INTERVIEW INSIGHT

Q. Why do some developers avoid the Repository Pattern with EF Core?



Because EF Core already provides abstraction over the database through DbContext and DbSet.

Adding another layer can sometimes create unnecessary complexity.



10. PRO TIP



Don't use patterns just because they're popular. Use them when they solve a **real problem**.



11. KEY INSIGHT

Architecture patterns are tools.

The goal isn't to use more patterns.

The goal is to solve problems effectively.



KEY TAKEAWAY

- ✓ The Repository Pattern helps organize data access logic and improve separation of concerns.
- ✓ It improves testability and maintainability in many applications.
- ✓ But with EF Core, it's important to understand when the pattern adds value and when it introduces unnecessary complexity.



NEXT UP (DAY 57)

Unit of Work Pattern in .NET – managing multiple database operations efficiently



SCENARIO FOR YOU

If EF Core already provides DbContext and DbSet, would you still introduce a Repository layer? Why or why not?



REMEMBER

- ✓ Understand the need
- ✓ Evaluate the trade-offs
- ✓ Keep it simple
- ✓ Solve real problems





Managing multiple database operations efficiently

**1. REAL SCENARIO**

A customer places an order.
Your application needs to:

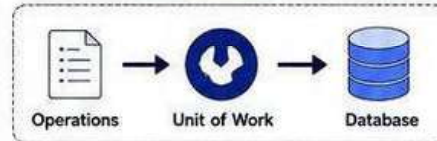
- ✓ Create an Order
- ✓ Add Order Items
- ✓ Update Inventory
- ✓ Create a Payment Record



What if one operation succeeds but another fails?
You could end up with inconsistent data.
That's where the Unit of Work Pattern comes in.

**2. WHAT IS THE UNIT OF WORK PATTERN?**

The Unit of Work Pattern treats multiple database operations as a single transaction.



Goal:

- ✓ Maintain data consistency
- ✓ Ensure all operations succeed together
- ✓ Roll back everything if something fails

3. EXAMPLE SCENARIO

Create Order

+



Add Order Items

+



Update Inventory

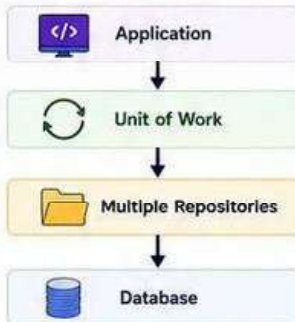
+



Create Payment



All changes
are committed
together.

4. TYPICAL FLOW**5. WHY TEAMS USE UNIT OF WORK**

- ✓ Transaction management
- ✓ Data consistency
- ✓ Centralized SaveChanges()
- ✓ Better control over database operations

**6. SIMPLIFIED EXAMPLE**

```

_unitOfWork.Orders.Add(order);
_unitOfWork.Payments.Add(payment);
await _unitOfWork.SaveChangesAsync();
  
```

Instead of saving each operation separately.

**7. COMMON PITFALL**

Calling `SaveChanges()` multiple times inside the same business workflow.

```

context.Orders.Add(order);
await context.SaveChangesAsync();
context.Products.Update(product);
await context.SaveChangesAsync();
...
  
```



Result:

- ✗ Partial updates
- ✗ Inconsistent data
- ✗ Harder error recovery

**8. PRODUCTION INSIGHT**

When using EF Core:
DbContext already behaves similarly to a Unit of Work.



In many modern applications
Use DbContext directly.



In other applications
Wrap it in a custom
Unit of Work abstraction.



The choice depends on architecture needs
and project complexity.

**9. PRO TIP**

Think in terms of:
Business Transactions
not
**Individual database
operations.**

**10. INTERVIEW INSIGHT**

Q. What is the difference between a
Repository and a Unit of Work?

Repository

Manages data access
for an entity.

VS

Unit of Work

Coordinates multiple
repositories and
commits changes
together.

**11. BENEFITS vs DRAWBACKS****BENEFITS**

- ✓ Consistent transactions
- ✓ Better data integrity
- ✓ Centralized commits
- ✓ Cleaner business workflows



VS

DRAWBACKS

- ✗ Additional abstraction
- ✗ Extra complexity in simple projects

**12. KEY INSIGHT**

The Unit of Work Pattern is
not about saving data.
It's about ensuring
related operations
succeed or fail
together.

**TAKEAWAY**

- ✓ Unit of Work helps manage multiple database operations as a single transaction.
- ✓ It ensures data consistency and keeps your application reliable.
- ✓ In EF Core, understanding how DbContext already provides similar behavior is just as important as understanding the pattern itself.

**NEXT UP (DAY 58)**

Dependency Injection in .NET –
how objects get created and
managed automatically

**SCENARIO FOR YOU**

If creating an order succeeds
but updating inventory fails,
should the order still exist in
the database?

**REMEMBER**

- ✓ Understand the need
- ✓ Apply the pattern correctly
- ✓ Keep it simple
- ✓ Solve real problems





Dependency Injection in .NET

How objects get created and managed automatically



1. REAL SCENARIO

Your service needs:

- ✓ A repository
- ✓ A logger
- ✓ A cache service
- ✓ An email service

Should the service create all these dependencies itself? 🤔

```
var repo = new UserRepository();
var logger = new Logger();
var cache = new CacheService();
var email = new EmailService();
```



This quickly becomes difficult to maintain and test.

That's where Dependency Injection comes in.



2. WHAT IS DEPENDENCY INJECTION?

Dependency Injection is a design pattern where dependencies are provided to a class instead of the class creating them itself.

Without DI (Tight Coupling)

```
public class UserService
{
    private readonly
    UserRepository _repo =
    new UserRepository();
}
```

With DI (Loose Coupling)

```
public class UserService
{
    private readonly
    IUserRepository _repo;

    public UserService(
        IUserRepository repo)
    { _repo = repo; }
}
```



Dependencies are injected, not created. This leads to loosely coupled and testable code.



3. DI FLOW IN .NET



The DI Container manages object creation and lifetime automatically.

4. SERVICE REGISTRATION

Register services in Program.cs / Startup.cs

```
builder.Services
    .AddScoped<IUserRepository,
    UserRepository>();

builder.Services
    .AddScoped<IUserService,
    UserService>();
```



.NET's built-in DI container manages object creation for you.



5. SERVICE LIFETIMES

SINGLETON



AddSingleton()

One instance for entire application.

Use for:

- Configuration
- Caching
- Shared services

SCOPED



AddScoped()

One instance per request (per scope).

Use for:

- DbContext
- Repositories
- Business services

TRANSIENT



AddTransient()

A new instance every time it is requested.

Use for:

- Lightweight services
- Stateless services

Choosing the right lifetime is important for performance and correctness.

6. COMMON PITFALL

Choosing the wrong lifetime.

Example:

```
builder.Services.AddSingleton<
    ApplicationDbContext>(); // ❌ Wrong!
```

Registering DbContext as Singleton can cause:

- ❌ Concurrency issues
- ❌ Memory problems
- ❌ Unexpected behavior



7. PRODUCTION INSIGHT

Most ASP.NET Core applications use:

- ✓ Scoped for DbContext
- ✓ Scoped for repositories
- ✓ Singleton for configuration services



The correct lifetime choice is critical for scalability.



8. PRO TIP

“Don't think of DI as a framework feature.

Think of it as a way to reduce dependencies between components.



9. INTERVIEW INSIGHT

Q. What is the difference between Singleton, Scoped, and Transient?

Lifetime	Description
Singleton	One instance for the entire application.
Scoped	One instance per request (per scope).
Transient	A new instance every time it is requested.



10. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Testability
- ✓ Flexibility
- ✓ Cleaner code
- ✓ Better maintainability
- ✓ Easier replacement of implementations



VS

DRAWBACKS

- ❌ More abstraction
- ❌ Can be confusing initially
- ❌ Incorrect lifetimes can cause bugs



11. KEY INSIGHT

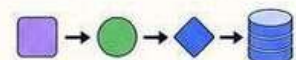


Dependency Injection isn't about creating objects. It's about managing dependencies effectively.



KEY TAKEAWAY

- ✓ Dependency Injection helps build loosely coupled and maintainable applications.
- ✓ It makes testing easier and code more flexible.
- ✓ Understanding service lifetimes is just as important as understanding DI itself.



NEXT UP (DAY 59)

Middleware in ASP.NET Core – how requests and responses flow through your application



SCENARIO FOR YOU

If you wanted to replace SQL Server with another database tomorrow, how much code would you need to change in your application?



REMEMBER

- ★ Understand the need
- ★ Use the right lifetime
- ★ Keep it simple
- ★ Solve real problems





Middleware in ASP.NET Core

How requests and responses flow through your application



1. REAL SCENARIO

A request reaches your API. Before it reaches your controller, you may want to:

- ✓ Log the request
- ✓ Authenticate the user
- ✓ Handle exceptions
- ✓ Validate headers
- ✓ Measure performance



Where should all this logic live?
That's where Middleware comes in.

2. WHAT IS MIDDLEWARE?

Middleware is software that sits in the request processing pipeline.

Every request passes through a series of middleware components before reaching your endpoint.



Each middleware can:

- ✓ Process the request
- ✓ Modify the request
- ✓ Pass it to the next middleware
- ✓ Stop the pipeline completely

3. ASP.NET CORE REQUEST PIPELINE



The request flows through each middleware in the order they are registered.

4. BUILT-IN MIDDLEWARE EXAMPLES

These are some commonly used built-in middleware:

```

app.UseAuthentication();
app.UseAuthorization();
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
  
```

These are all middleware components.

5. WHY TEAMS USE MIDDLEWARE

- ✓ Centralized logic
- ✓ Cleaner controllers
- ✓ Better reusability
- ✓ Consistent request handling
- ✓ Cross-cutting concerns in one place



Result:

Less duplication and more maintainable code.

6. SIMPLE CUSTOM MIDDLEWARE

Example of a basic custom middleware:

```

public async Task InvokeAsync(
    HttpContext context,
    RequestDelegate next)
{
    Console.WriteLine("Request Started");

    await next(context);

    Console.WriteLine("Request Finished");
}
  
```

7. COMMON PITFALL (MIDDLEWARE ORDER)

Middleware order matters.

Example:



Can lead to:

- ✗ Failed authorization
- ✗ Unexpected behavior
- ✗ Security issues



8. PRODUCTION INSIGHT

Many production systems use middleware for:

- ✓ Global exception handling
- ✓ Request logging
- ✓ Correlation IDs
- ✓ Performance monitoring
- ✓ Security headers



Instead of duplicating the same code across controllers.

9. PRO TIP

Keep middleware focused on one responsibility.

GOOD MIDDLEWARE

- ✓ Logging
- ✓ Exception handling
- ✓ Authentication
- ✓ Request validation

AVOID

- ✗ Creating one giant middleware that does everything.



10. INTERVIEW INSIGHT

Q. What happens if middleware does NOT call:

`await next(context);`



The pipeline stops.
The request never reaches the next middleware or controller.



11. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Reusable
- ✓ Centralized logic
- ✓ Cleaner code
- ✓ Better maintainability



VS

DRAWBACKS

- ✗ Debugging order issues can be difficult
- ✗ Poorly designed middleware can affect performance



12. KEY TAKEAWAY

- ✓ Middleware processes requests and responses in ASP.NET Core.
- ✓ The order of middleware is just as important as the middleware itself.



NEXT UP (DAY 60)

Custom Middleware in ASP.NET Core – building your own request pipeline components



SCENARIO FOR YOU

If every controller needs request logging, would you duplicate the code everywhere or place it in middleware?



REMEMBER

- ✓ Understand the pipeline
- ✓ Use the right order
- ✓ Keep it simple & focused
- ✓ Test your middleware





Building your own request pipeline components



1. REAL SCENARIO

Every API request should:

- ✓ Log request details
- ✓ Track execution time
- ✓ Add correlation IDs
- ✓ Validate custom headers

Should this code be copied into every controller? 🤔



Of course not.
That's where
Custom Middleware
becomes powerful.

2. WHAT IS CUSTOM MIDDLEWARE?

Custom Middleware allows you to add your own logic into the ASP.NET Core request pipeline.



Your middleware can:

- ✓ Execute code before the request
- ✓ Execute code after the response
- ✓ Modify requests and responses
- ✓ Stop the pipeline if needed

3. REQUEST FLOW



The request flows through each component in the order they are registered.

4. SIMPLE CUSTOM MIDDLEWARE

```

public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;

    public RequestLoggingMiddleware(
        RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(
        HttpContext context)
    {
        Console.WriteLine("Request Started");

        await _next(context); // Call next middleware

        Console.WriteLine("Request Finished");
    }
}
  
```

5. REGISTER MIDDLEWARE

Register your middleware in Program.cs (in the correct order).

```
app.UseMiddleware<RequestLoggingMiddleware>();
```

- ✓ Once registered, every request passes through it.

6. WHY TEAMS CREATE CUSTOM MIDDLEWARE

- ✓ Request logging
- ✓ Global exception handling
- ✓ Correlation IDs
- ✓ Security checks
- ✓ Performance monitoring
- ✓ Multi-tenant validation

7. COMMON PITFALL

Doing heavy business logic inside middleware.

Example:

- ✗ Database queries
- ✗ Complex business rules
- ✗ Long-running operations



Middleware should stay focused on cross-cutting concerns.

8. PRODUCTION INSIGHT

Large enterprise applications often use middleware for:

- 🔍 Distributed tracing
- 🔗 Request correlation
- 🔒 Security auditing
- 📊 API monitoring



📌 Because these concerns apply to every request.

9. PRO TIP

Ask yourself:

"Will this logic be needed by almost every request?"

↓
If yes,
Middleware might be
the right place.



10. INTERVIEW INSIGHT

Q. What is the difference between Middleware and Filters?

Middleware	Filters
Works for every request in the pipeline.	Works only within MVC / Web API actions.
Executes earlier in the request lifecycle.	Executes later, around the action execution.
Configured in Program.cs using UseMiddleware().	Configured using attributes or options.

11. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Reusable
- ✓ Centralized logic
- ✓ Cleaner controllers
- ✓ Easy monitoring



VS

DRAWBACKS

- ✗ Wrong placement can hurt performance
- ✗ Debugging order issues can be tricky



12. KEY INSIGHT

Great middleware focuses on infrastructure concerns, not business logic.



KEY TAKEAWAY

- ✓ Custom Middleware allows you to inject reusable logic into the ASP.NET Core request pipeline.
- ✓ Use it for cross-cutting concerns that apply across your application.



NEXT UP (DAY 61)

Global Exception Handling in ASP.NET Core - handling errors consistently across your APIs



SCENARIO FOR YOU

If every API endpoint needs request logging, would you implement it in every controller action or once in middleware?



REMEMBER

- ✦ Keep middleware focused
- ✦ Register in the correct order
- ✦ Call next(context)
- ✦ Test your middleware





Handling errors consistently across your APIs

1. REAL SCENARIO

A user calls your API.

Something goes wrong:

- Database connection fails
- Null reference exception occurs
- External service is unavailable



Should every controller action have its own try-catch block?

Of course not.

That's where Global Exception Handling comes in.

2. WHAT IS GLOBAL EXCEPTION HANDLING?

Global Exception Handling allows your application to catch and handle unhandled exceptions from a central place.

Without Global Handling

```
try
{
    // Business Logic
}
catch(Exception ex)
{
    // Handle Error
}
```

In every action ❌

With Global Handling

Handle errors once for the entire application



3. ERROR HANDLING FLOW



All unhandled exceptions are caught and handled in one central place.

4. STANDARD ERROR RESPONSE EXAMPLE

Clients receive predictable error responses.

```
{
  "statusCode": 500,
  "message": "An unexpected error occurred",
  "timestamp": "2024-06-08T09:15:30Z",
  "path": "/api/users"
}
```



Expose safe, meaningful information to clients, log detailed errors internally.



5. WHY TEAMS USE IT

- Centralized error management
- Cleaner controllers
- Better observability
- Easier troubleshooting
- Consistent API behavior



Better for developers, better for users, better for the entire system.



6. COMMON PITFALL

Returning internal exception details to users.

- Stack traces
- Database connection strings
- Sensitive implementation details
- System information



Always log detailed errors internally, but return safe messages externally.



7. PRODUCTION INSIGHT

Many production APIs combine:

- Global Exception Middleware
- Structured Logging
- Correlation IDs
- Monitoring Tools



Helps quickly diagnose issues and reduce MTTR.



8. PRO TIP

Handle exceptions at the highest appropriate level.

```
app.UseExceptionHandler();
app.UseMiddleware<ExceptionMiddleware>();
```



Avoid wrapping every method with unnecessary try-catch blocks.



9. INTERVIEW INSIGHT

Q. Why use Global Exception Handling instead of try-catch everywhere?

It reduces duplication, improves consistency, and centralizes error management.



10. BENEFITS vs DRAWBACKS

BENEFITS

- Consistent error responses
- Cleaner code
- Easier maintenance
- Better logging
- Improved reliability



VS

DRAWBACKS

- Poor implementation can hide useful debugging information
- Requires proper logging strategy



11. KEY INSIGHT

Good APIs don't just handle success scenarios.

They handle failure scenarios gracefully too.



KEY TAKEAWAY

- Global Exception Handling provides a centralized way to manage errors across your application.
- Consistent error handling improves reliability, maintainability, and user experience.



NEXT UP (DAY 62)

Logging in ASP.NET Core - tracking what really happens inside your application



SCENARIO FOR YOU

If 50 API endpoints can throw exceptions, would you rather maintain 50 try-catch blocks or one centralized error handling strategy?



REMEMBER

- Log the exception (details internally)
- Return safe message to users
- Keep responses consistent
- Monitor and improve





Logging in ASP.NET Core

Tracking what really happens inside your application



1. REAL SCENARIO

A user reports:

"The API is not working."

But...

- ❌ No error screenshot
- ❌ No reproduction steps
- ❌ No clear details



How would you investigate what happened?

That's where Logging becomes essential.



2. WHAT IS LOGGING?

Logging is the process of recording application events while your system is running.

Logs help answer questions:

- ✓ What happened?
- ✓ When did it happen?
- ✓ Why did it happen?
- ✓ Which request caused it?



If you can't see what's happening inside your application, you can't effectively support it.



3. LOGGING FLOW



Logs provide visibility into your application and help you troubleshoot issues faster.



4. ILogger EXAMPLE

ASP.NET Core provides built-in logging through `ILogger<T>`.

```

public class HomeController
{
    private readonly ILogger<HomeController> _logger;

    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    public async Task ProcessUser()
    {
        _logger.LogInformation("Processing user request.");
    }
}
  
```



Inject `ILogger<T>` via Dependency Injection.



5. LOG LEVELS EXPLAINED

Trace	Very detailed diagnostics
Debug	Development troubleshooting
Information	Normal application events
Warning	Unexpected but recoverable situations
Error	Operation failed
Critical	Application-threatening failures



Choose the right level to capture the right amount of detail.



6. COMMON PITFALL

Logging too much information.

- ❌ Passwords
- ❌ Access tokens
- ❌ Credit card details
- ❌ Sensitive customer data



Logs should help debugging, not create security risks.



7. PRODUCTION INSIGHT

Many production systems generate thousands of log entries every minute.

Good logging should be:

- ✓ Meaningful
- ✓ Searchable
- ✓ Structured
- ✓ Actionable



Good logs tell a story. Bad logs create confusion.



8. PRO TIP - STRUCTURED LOGGING

Instead of:

```
_logger.LogInformation("User created");
```



Prefer:

```
_logger.LogInformation(
    "User {userId} created successfully",
    userId);
```



Structured logs are easier to search, filter and analyze.



9. INTERVIEW INSIGHT

Q. What is the difference between Information and Error?

Information	Error
Normal application behavior.	An operation failed and requires attention.
Example: User logged in successfully.	Example: Failed to connect to database.



10. SENIOR DEVELOPER NOTE

The challenge is not generating logs.

The challenge is generating logs that are useful during production incidents.



11. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Faster troubleshooting
- ✓ Better monitoring
- ✓ Easier debugging
- ✓ Production visibility



VS

DRAWBACKS

- ❌ Excessive logging can impact performance
- ❌ Poor logs create noise instead of insight



12. KEY TAKEAWAY

Logging helps track, troubleshoot, and monitor application behavior.

Good logs tell a story. Bad logs create confusion.



NEXT UP (DAY 63)

Setting up .NET - structured logging for modern applications



SCENARIO FOR YOU

A production issue occurred at 2 AM. Would you rather investigate with detailed logs, or rely on user complaints?



REMEMBER

- ✓ Log the right things
- ✓ Use the right level
- ✓ Keep logs structured
- ✓ Protect sensitive data



TOOLS YOU CAN USE

Built-in Logger
Serilog
Slog
Application Insights
Seq / Kibana / ELK Stack



Serilog

Serilog in .NET

Structured logging for modern applications



1. REAL SCENARIO

Your production system generates thousands of logs every day.

You need to answer questions like:

- Which user triggered the error?
- Which API failed?
- How many failures happened today?
- Which requests took the longest?



Searching through plain text logs can quickly become difficult.

That's where Serilog becomes powerful.

2. WHAT IS SERILOG?

Serilog is a popular structured logging library for .NET.

Instead of storing logs as plain text:

Plain Text Log

User created successfully

Serilog stores structured data:

Structured Log (JSON)

```
{
  "Timestamp": "2024-06-16T09:30:15Z",
  "Level": "Information",
  "Message": "User created successfully",
  "UserId": 101,
  "Action": "CreateUser",
  "Status": "Success"
}
```

This makes logs easier to:



Search



Filter



Analyze



Monitor

3. LOGGING FLOW



4. PLAIN TEXT vs STRUCTURED LOGGING

Plain Text Logging

```
2024-06-16 09:30:15 INFO
User: 101 created
2024-06-16 09:30:16 ERROR
Something went wrong
```

- Hard to search
- Hard to filter
- Not machine-friendly
- Limited analysis

Structured Logging (Serilog)

```
{
  "Timestamp": "2024-06-16T09:30:15Z",
  "Level": "Information",
  "Message": "User created",
  "UserId": 101
}, {
  "Timestamp": "2024-06-16T09:30:16Z",
  "Level": "Error",
  "Message": "Something went wrong",
  "UserId": 101
}
```

- Easy to search
- Easy to filter
- Machine-friendly
- Powerful analysis

5. INSTALL & CODE EXAMPLE

Install Serilog

`Install-Package Serilog,AspnetCore`

Program.cs

```
using Serilog;

var builder = WebApplication.CreateBuilder(args);
Log.Logger = new LoggerConfiguration()
    .WriteTo.Console()
    .CreateLogger();
builder.Host.UseSerilog();

var app = builder.Build();
```

Without Serilog (Plain Text)

```
// Not structured
Log.Logger.LogInformation(
    "User created");
```

With Serilog (Structured)

```
// Structured Log
Log.Information(
    "User {userId} created successfully", userId);
```

Structured logs capture properties as fields, not just text.



6. COMMON PITFALL

Logging entire objects blindly.

Example:

```
Log.Information("{0}", user);
```

Can accidentally expose:

- Passwords
- Tokens
- Sensitive customer data
- Internal system details



Always review what gets logged.
Log only what is necessary.



7. PRODUCTION INSIGHT

Many enterprise applications send Serilog logs to:

Elasticsearch

Kibana

SEO: Seq

Azure Application Insights

Splunk

This enables powerful search, dashboards, alerts, and monitoring capabilities.

8. PRO TIP

Use structured properties instead of string concatenation.

❌ Avoid this:

```
Log.Information("User { } = {userId} + created successfully");
```

✅ Do this instead:

```
Log.Information("User {userId} created successfully", userId);
```



Structured properties make querying logs much easier.

9. INTERVIEW INSIGHT

Q. Why is structured logging better than plain text logging?

Because log properties become searchable and filterable, making troubleshooting much faster.



10. SERILOG DEVELOPER NOTE

Logs are not just for debugging in modern systems, logs are a critical part of observability and production monitoring.



Good logs + Metrics + Traces = Complete Observability

11. BENEFITS vs DRAWBACKS

BENEFITS

- Better searchability
- Rich monitoring
- Easier diagnosis
- Production visibility
- Global ecosystem

DRAWBACKS

- More storage usage
- Poor logging practices can still create noise

VS



12. KEY TAKEAWAY

- Serilog enables structured logging in .NET applications.
- Well-structured logs help teams troubleshoot issues faster.
- The value of logs comes from being able to find useful information when something goes wrong.



Your Application

Serilog

Logs

Better Insights & Faster Actions



NEXT UP (DAY 64)

Authentication vs Authorization - understanding the difference



SCENARIO FOR YOU!

If an error happens for a specific customer at 2 PM, would you rather search millions of text logs or filter logs by CustomerId in seconds?



REMEMBER

- Log the right things
- Join the right levels
- Keep logs structured
- Protect sensitive data



TOOLS YOU CAN USE

Seq
Splunk
Elastic Stack (ELK)
Azure Application Insights
Datadog / New Relic



Authentication vs Authorization

Understanding the difference



1 REAL SCENARIO

A user tries to access your banking application.
Before allowing access, the system needs to answer two questions:

- 1 Who are you?
- 2 What are you allowed to do?

These may sound similar, but they solve completely different problems.

That's where Authentication and Authorization come in.

2 WHAT IS AUTHENTICATION?

Authentication answers:
"Who are you?"

The system verifies the user's identity.

Examples:

- 1 Username & Password
- 2 OTP (One Time Password)
- 3 Biometric Login
- 4 Social Login (Google, Microsoft, etc.)

Goal: Verify the identity of the user.

3 WHAT IS AUTHORIZATION?

Authorization answers:
"What are you allowed to do?"

After identity is verified, the system checks permissions.

Examples:

- 1 Admin can create users
- 2 Manager can approve requests
- 3 Customer can view only their own data

Goal: Verify permissions and access.

4 REQUEST FLOW



5 REAL EXAMPLE

Imagine:
Srinja logs into an application.



- Authentication**
 - Confirms Srinja is a valid user.
- Authorization**
 - Allows access to user features.
 - Blocks access to Admin-only endpoints.

6 WHY BOTH ARE NEEDED



Without Authentication:

- 1 Anyone can pretend to be a user.



Without Authorization:

- 1 Every authenticated user gets access to everything.



Both are essential for security.

7 COMMON PITFALL

Assuming authentication automatically provides authorization.

Example:

User logged in
(Authentication successful)



User can access
all APIs
(Authorization)



Permissions still need to be validated for each resource or action.

8 PRODUCTION INSIGHT

Many security incidents happen because:

- 1 Authentication implemented
 - 2 Authorization rules incomplete
- Proper permission checks are just as important as login functionality.

9 PRO TIP

Think of it this way:



10 INTERVIEW INSIGHT

Q. What comes first: Authentication or Authorization?



Authentication always comes first.

The system must know who you are before deciding what you're allowed to do.

11 SENIOR DEVELOPER NOTE



In enterprise systems, authorization often becomes more complex than authentication because permissions can vary by:

- 1 Role
- 2 Department
- 3 Region
- 4 Resource ownership
- 5 Business rules

12 AUTHENTICATION vs AUTHORIZATION

AUTHENTICATION	AUTHORIZATION
1 Verifies identity	1 Verifies permissions
2 Login process	2 Access control
3 Answers "Who are you?"	3 Answers "What can you do?"

13 KEY INSIGHT

Authentication and Authorization work together.



One verifies identity.
The other protects access.

14 KEY TAKEAWAY



Authentication verifies who the user is.



Authorization determines what the user is allowed to do.



Secure applications require both.



NEXT UP (DAY 65)
JWT Authentication in ASP.NET Core - securing APIs with tokens



SCENARIO FOR YOU
If a user successfully logs in, should they automatically gain access to every feature in the application?



REMEMBER

- 1 Authenticate the user
- 2 Authorize the actions
- 3 Always validate permissions



TOOLS YOU WILL USE
ASP.NET Identity
JWT Tokens
Roles & Claims
Policy Based Authorization

JWT Authentication

in ASP.NET Core



Securing APIs with Tokens

1 REAL SCENARIO

A user logs into your application.
After login:

- ✓ The user accesses APIs
- ✓ Mobile apps make requests
- ✓ Frontend calls backend services

How does the API know the user is authenticated without asking for username and password on every request?

That's where JWT Authentication comes in.

2 WHAT IS JWT?

JWT stands for JSON Web Token. It is a compact token used to securely transmit user identity information between systems.



JWT allows stateless authentication between client and server.

3 JWT AUTHENTICATION FLOW



4 AUTHORIZATION HEADER EXAMPLE

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

The token travels with each request.



Bearer Token = Proof of identity for accessing APIs

5 WHAT'S INSIDE A JWT?

A JWT typically contains claims about the user and token information.

```
{
  "sub": "1234",
  "name": "John Doe",
  "role": "Admin",
  "email": "john.doe@acme.com",
  "iat": 1516239024,
  "exp": 1716239024
}
```

- ✓ User ID (sub)
- ✓ Username (name)
- ✓ Role (role)
- ✓ Claims
- ✓ Issued At (iat)
- ✓ Expiration Time (exp)



The payload is encoded, not encrypted by default.

6 WHY TEAMS USE JWT

- ✓ Stateless authentication
- ✓ Scalable APIs
- ✓ Mobile app support
- ✓ Microservices friendly
- ✓ No server-side session storage
- ✓ Works across different domains



Perfect for modern distributed and cloud-based applications.

7 COMMON PITFALL

Storing sensitive information inside JWTs.

- ✗ JWTs are encoded
- ✗ JWTs are NOT encrypted by default
- ✗ Never store sensitive data such as:



Anyone can decode the token and view the payload.

8 PRODUCTION INSIGHT

Many production systems use a combination of:

- ⌚ Short-lived Access Tokens (5-15 minutes)
- 🔄 Refresh Tokens (Long-lived)
- 📅 Token expiration policies



This improves security while maintaining a good user experience.

9 PRO TIP

Always validate tokens properly.

- ✓ Validate expiration time
- ✓ Validate issuer (iss)
- ✓ Validate audience (aud)
- ✓ Validate signing key
- ✓ Use strong signing algorithms



Never accept tokens without proper validation.

10 INTERVIEW INSIGHT

Q. Why is JWT called "stateless"?

Because the server does not need to store session information.

The token itself contains the identity information required for authentication.



Server validates the token on every request without storing any session.

11 SENIOR DEVELOPER NOTE

Implementing JWT authentication is easy.

Managing token expiration, refresh tokens, revocation, and security policies is where real-world complexity begins.



12 BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Stateless
- ✓ Mobile-friendly
- ✓ Works well with APIs
- ✓ No server-side session storage

VS

DRAWBACKS

- ✗ Token revocation can be challenging
- ✗ Poor implementation creates security risks
- ✗ Larger payloads increase token size



KEY TAKEAWAY



JWT enables secure, stateless authentication for modern applications.

JWT is not just a token. It's a security mechanism that allows APIs to verify identity without maintaining server-side sessions.



Proper token validation and expiration management are critical for security.

13 NEXT UP (DAY 66)

Refresh Tokens - keeping users logged in securely



14 SCENARIOS FOR YOU

Would you prefer users to log in every 15 minutes, or use a secure refresh token strategy to maintain a good user experience?



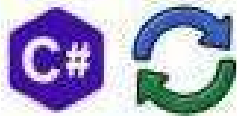
15 REMEMBER

- ✓ Secure the token
- ✓ Validate the token
- ✓ Expire the token
- ✓ Refresh the token



16 TOOLS YOU CAN USE

Microsoft.IdentityModel.Tokens
System.IdentityModel.Tokens.Jwt
ASP.NET Core Authentication
Fiddler / Wireshark for testing



Refresh Tokens in ASP.NET Core

Keeping users logged in securely



1 REAL SCENARIO

Your application uses JWT Authentication.

For security reasons:

- Access Tokens expire in 15 minutes.

After 15 minutes:

- Users would need to log in again.

How do modern applications keep users logged in securely?

That's where Refresh Tokens come in.

2 WHAT IS A REFRESH TOKEN?

A Refresh Token is a long-lived credential used to obtain a new Access Token after the current one expires.



3 AUTHENTICATION FLOW



4 ACCESS TOKEN vs REFRESH TOKEN

Access Token	Refresh Token
Short-lived (5 - 30 Minutes)	Long-lived (Days or Weeks)
Used for API requests	Used only to obtain new Access Tokens
Sent with every API request	Stored securely, sent only when Access Token expires
Goal: Access protected resources	Goal: Get new Access Token

5 WHY NOT USE LONG-LIVED JWTs?

Example: JWT Valid For 30 Days

Problem:

- If stolen, attacker gets access for 30 days.

Instead:

- Short Access Token
- Long Refresh Token



Short-lived Access Token + Long Refresh Token provides better security.

6 COMMON PITFALL

Storing Refresh Tokens insecurely.

- Local storage without protection
- Exposed client-side storage
- No token rotation
- Not revoking on logout

Refresh Tokens should be treated like sensitive credentials.

7 PRODUCTION INSIGHT

Most production systems implement:

- Refresh Token Rotation
- Token Revocation
- Device Tracking
- Logout Invalidation

These measures significantly reduce the impact of token theft.

8 PRO TIP

When issuing a new Refresh Token:

Invalidate the old one.



This is called:

Refresh Token Rotation

and greatly improves security.

9 INTERVIEW INSIGHT

Q: Why not simply increase JWT expiration time?



Because longer-lived tokens increase the impact of token theft.

Short-lived Access Tokens reduce risk and limit the damage.

10 SENIOR DEVELOPER NOTE



Generating tokens is easy. Designing a secure token lifecycle - including rotation, revocation, expiration, and logout behavior - is where real authentication systems become challenging.

Security is in the lifecycle, not just in the token.

11 BENEFITS vs DRAWBACKS

BENEFITS

- Better user experience
- Improved security
- Reduced login frequency
- Works well with modern APIs



VS

DRAWBACKS

- Additional implementation complexity
- Requires secure storage
- Token lifecycle management becomes important



12 KEY TAKEAWAY



- Access Tokens should be short-lived.
- Refresh Tokens allow secure renewal of expired Access Tokens.
- Proper rotation and revocation strategies are critical.

Refresh Tokens balance security and usability. Users stay logged in without sacrificing security.



13 NEXT UP (DAY 67)

Role-Based Authorization in ASP.NET Core - controlling access with roles



SCENARIO FOR YOU

If a user's token is stolen, would you rather it remain valid for 30 days or expire in 15 minutes?



REMEMBER

- Short Access Token
- Long Refresh Token
- Rotate & Revoke
- Stay Secure



TOOLS YOU CAN USE

ASP.NET Core Identity
JWT (System.IdentityModel.Tokens.Jwt)
Refresh Token Rotation
Redis / DB for token storage



Role-Based Authorization in ASP.NET Core

Controlling access with roles



1 REAL SCENARIO

Your application has three types of users:

- Admin
- Manager
- Customer

Should every user be allowed to:

- Delete users?
- View financial reports?
- Manage system settings?



Of course not.
That's where
Role-Based Authorization
comes in.

2 WHAT IS ROLE-BASED AUTHORIZATION?

Role-Based Authorization restricts access based on a user's assigned role.

Instead of checking every user individually:



The application decides what actions are allowed based on roles.

3 AUTHORIZATION FLOW



The API checks the role in the token and allows or denies access based on role permissions.

4 EXAMPLE ROLES AND THEIR PERMISSIONS

ADMIN	MANAGER	CUSTOMER
- Manage users - View reports - Configure system settings - Access all features	- View reports - Manage team data - Approve requests - Control manage system settings	- View own data - Update profile - Create requests - Cannot access admin features
Full access to application features.	Limited access based on responsibilities.	Access only to personal and allowed features.

5 ASP.NET CORE EXAMPLE

Authorize only users with the "Admin" role.

```

[Authorize(Roles = "Admin")]
public IActionResult DeleteUser(int id)
{
    // Only Admin users can access
    return Ok(new { Message = "User deleted" });
}
  
```



Only users with the Admin role in their token can access this endpoint.

6 WHY TEAMS USE ROLE-BASED AUTHORIZATION

- Simpler permission management
- Better security
- Easier maintenance
- Consistent access control



Roles help enforce permissions across the entire application.

7 COMMON PITFALL

Relying only on frontend restrictions.



Hiding Admin button in UI is not enough.



Always enforce authorization on the server side.

8 PRODUCTION INSIGHT

As applications grow:

- Too many roles become difficult to manage.

Many enterprise systems eventually move toward:

- Claims-Based Authorization
- Policy-Based Authorization for more flexibility and scalability



9 PRO TIP

Roles should represent job responsibilities, not individual users.

Good Examples

- Admin
- Manager
- SupportAgent
- FinanceUser



Avoid

- Bob_Smith
- John_Doe123
- Admin_John
- User_304



10 INTERVIEW INSIGHT

Q. What's the difference between Authentication and Authorization?



Authentication → Verifies identity (Who are you?)



Authorization → Verifies permissions (What are you allowed to do?)



Role-Based Authorization is one way of implementing Authorization.

11 SENIOR DEVELOPER NOTE



Role-Based Authorization works well initially.

However, large enterprise systems often require more granular control than roles alone can provide.



Keep your authorization model flexible for future growth.

12 BENEFITS vs DRAWBACKS

BENEFITS

- Easy to understand
- Easy to implement
- Strong security foundation
- Works well for many applications

VS

DRAWBACKS

- Can become complex with many roles
- Less flexible than policy-based approaches

KEY TAKEAWAY



Authentication answers: "Who are you?"



- Role-Based Authorization controls access using assigned roles.
- Authorization must always be enforced on the server side.
- Roles work well for many applications but may evolve into more advanced authorization models as systems grow.



Authorization answers: "What are you allowed to do?"



NEXT UP (DAY 68)
API Security Best Practices - protecting your APIs beyond authentication



SCENARIO FOR YOU
If a user manually calls an Admin API endpoint from Postman, should hiding the Admin button in the UI be enough to stop them?



REMEMBER

- Authentication is not Authorization
- Authorization is not Authentication
- Always enforce authorization on the server side



TOOLS YOU CAN USE
ASP.NET Core Identity, JWT, System.IdentityModel.Tokens.Jwt, Authorization Policies, RoleManager, UserManager





API SECURITY BEST PRACTICES

Protecting your APIs from common threats



1. REAL SCENARIO

Your API is live in production.
It handles:

- ✓ User accounts
- ✓ Payments
- ✓ Orders
- ✓ Sensitive business data

What if someone tries to:

- ✗ Access data without authorization
- ✗ Abuse your endpoints
- ✗ Steal tokens
- ✗ Flood your API with requests

- ✓ API Security is not optional.
It's a core part of backend engineering.



2. WHY API SECURITY MATTERS

Even a well-designed API can become vulnerable if security is ignored.

Security helps protect:



Users



Data



Business
Operations



System
Availability



3. SECURITY LAYERS (DEFENSE IN DEPTH)



Multiple layers work together to secure your API.



4. TOP 5 API SECURITY BEST PRACTICES



1. ALWAYS USE HTTPS



HTTP
Data travels
unencrypted



HTTPS
Data is encrypted
in transit



Protects credentials
and tokens.



2. SECURE AUTHENTICATION

Use trusted approaches
like:

- JWT
- OAuth 2.0
- OpenID Connect

✗ **Application**
authentication systems
unless absolutely
necessary.



3. VALIDATE INPUT

Never trust user input.

Example risks:

- SQL Injection
- Malicious payloads
- Invalid data

Always:

- ✓ Validate
- ✓ Sanitize
- ✓ Apply constraints



4. IMPLEMENT RATE LIMITING

Protect APIs from:

- Abuse
- Bots
- Traffic spikes

Example:

100 requests / minute

per user.



5. PROTECT SENSITIVE DATA

Never expose:

- ✗ Passwords
- ✗ Secrets
- ✗ Internal connection strings
- ✗ Detailed exception information

- ✓ Return only what's
necessary



5. COMMON PITFALL

Relying only on authentication.
Many developers think:

User is logged in
=
User can access everything



Not true.

- ✗ Authentication verifies identity.
- ✗ Authorization verifies permissions.



6. PRODUCTION INSIGHT

Most real-world security incidents happen because of:



✗ Misconfiguration



✗ Excessive permissions



✗ Exposed secrets



✗ Weak validation



Not because hackers "broke encryption."
But because small gaps create big risks.



7. PRO TIP

Apply the
Principle of Least Privilege.

Users and services
should only receive
the permissions they
actually need.



Minimum access. Maximum security.



8. INTERVIEW INSIGHT

Q. What is the difference between
Authentication and Authorization?



Authentication
Who are you?



Authorization
What are you
allowed to do?



Both are required.



9. BENEFITS vs RISKS

STRONG SECURITY

- ✓ Protects users
- ✓ Protects data
- ✓ Improves trust
- ✓ Reduces incidents



VS

POOR SECURITY

- ✗ Data leaks
- ✗ Account compromise
- ✗ Financial loss
- ✗ Reputation damage



10. KEY INSIGHT



Security is not
a feature you
add later.
It should be
considered from
the beginning of
API design.



KEY TAKEAWAY

- ✓ Secure APIs use multiple layers of protection, including HTTPS, authentication, authorization, validation, and rate limiting.
- ✓ Good security is about reducing risk, not eliminating it completely.



NEXT UP (DAY 69)

API Versioning in ASP.NET Core –
evolving APIs without breaking
clients



SCENARIO FOR YOU

If an authenticated user tries to
access another user's sensitive
data, what should stop them:
authentication, authorization,
or both?



REMEMBER

- ✗ Use HTTPS everywhere
- ✗ Validate every input
- ✗ Follow least privilege
- ✗ Log & monitor continuously





API VERSIONING IN ASP.NET CORE

Evolving APIs without breaking clients



1. REAL SCENARIO

Your mobile app is using:

GET /api/products

Thousands of users depend on it.

Now your team needs to:

- ✓ Add new fields
- ✓ Change response structure
- ✓ Introduce new functionality

What happens if you simply change the existing API?

Existing clients may break. 🚫



That's where
API Versioning
becomes important.



2. WHAT IS API VERSIONING?

API Versioning allows multiple versions of an API to coexist.

Example:

GET /api/v1/products (Old Client)

GET /api/v2/products (New Client)

This allows older clients to continue working while newer clients use updated functionality.



3. WHY VERSION APIS?

Without versioning:



With versioning:



4. COMMON VERSIONING STRATEGIES

A. URL VERSIONING (MOST COMMON)

Include version in the URL path.

GET /api/v1/products

GET /api/v2/products

- ✓ Easy to understand and cache-friendly.

B. QUERY STRING VERSIONING

Pass version as a query parameter.

GET /api/products?version=1

- ✓ Simple to implement.

C. HEADER VERSIONING

Pass version in the request header.

Api-Version: 1.0

- ✓ Keeps URL clean.
- ✗ Not easily visible in browser.



5. ASP.NET CORE EXAMPLE

```
[ApiVersion("1.0")]
Route("/api/v{version}/products")
public class ProductsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetProducts(V1)
    {
        // v1 logic
        return Ok(new { Message = "V1 Products" });
    }
}
```

- 1 Make sure to add API Versioning packages from NuGet and configure in Program.cs



6. COMMON PITFALL

Making breaking changes without versioning.

Example:

V1 Response

```
{
  "name": "Laptop"
}
```

V2 Response (Breaking Change)

```
{
  "productName": "Laptop"
}
```

- ✗ Older clients may fail unexpectedly.



7. PRODUCTION INSIGHT

Many large organizations maintain multiple API versions at the same time.

v1 v2 v3

Because not every client upgrades immediately. Backward compatibility is a business decision, not just a technical one.

- ★ Plan deprecation, communicate early and give enough time for migration.



8. PRO TIP

Version APIs when introducing breaking changes.

Don't create a new version for every small enhancement.



9. INTERVIEW INSIGHT

Q. What is a breaking change?

Examples:

- 1 Renaming fields
- 2 Removing endpoints
- 3 Changing response contracts
- 4 Changing data types (incompatible)



10. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Backward compatibility
- ✓ Safer deployments
- ✓ Easier evolution
- ✓ Better client support



DRAWBACKS

- ✗ Additional maintenance
- ✗ Multiple versions to support
- ✗ More testing effort



VS



11. KEY INSIGHT

Great APIs evolve.

Great APIs evolve without breaking existing clients.



API Versioning is often less about technology and more about protecting client applications.



KEY TAKEAWAY

- ✓ API versioning allows systems to evolve safely while maintaining compatibility for existing consumers.
- ✓ Versioning is about reducing risk, not adding complexity.



NEXT UP (DAY 70)

BEST API Best Practices – designing APIs that are clean, scalable, and easy to consume



SCENARIO FOR YOU

If a mobile application depends on your API, would you rather force every user to update immediately or support multiple API versions during the transition?



REMEMBER

- Plan your version.
- Communicate changes.
- Deprecate gracefully.
- Help your clients succeed.





REST API BEST PRACTICES

Designing APIs that are clean, scalable, and easy to consume



1. REAL SCENARIO

Two APIs provide the same functionality.

API A (Hard to Understand)

```
GET /GetAllUsersData
POST /CreateNewUserRecord
GET /FetchUserById
```



API B (Easy to Understand)

```
GET /users
POST /users
GET /users/{id}
```



Clean and consistent APIs are easier to understand and maintain.



2. WHAT IS REST?

REST (Representational State Transfer) is an architectural style for designing web APIs.

A good REST API should be:



Predictable



Consistent



Easy to Consume



Easy to Maintain



3. HTTP METHODS (Use Properly)

Method	Purpose	Example
GET	Retrieve data	GET /products
POST	Create new resource	POST /products
PUT	Update existing resource	PUT /products/{id}
DELETE	Delete resource	DELETE /products/{id}



Use the correct HTTP method for the correct intention.

4. TOP 5 REST API BEST PRACTICES



1. USE NOUNS, NOT VERBS



Bad

```
GET /GetProducts
POST /CreateProduct
```



Better

```
GET /products
POST /products
```



HTTP methods already describe the action.



2. USE PROPER HTTP METHODS



/products



/products



/products/{id}



/products/{id}



Each method has a specific purpose.



3. RETURN MEANINGFUL STATUS CODES



OK



Created



Bad Request



Unauthorized



Not Found



Internal Server Error



Status codes help clients understand what happened.



4. KEEP URLS SIMPLE



Bad

```
/api/get-all-active-products-list
```



Better

```
/api/products
```



Simple URLs are easier to read and remember.



5. USE PAGINATION



Avoid

```
GET /products
(Returns 100,000 records)
```



Better

```
GET /products?page=1&page_size=20
```



Pagination improves performance and user experience.

5. REST REQUEST/RESPONSE FLOW



Client

Request



API

Response



Product List



Client sends a request. API processes it and returns the appropriate response.



COMMON PITFALL

Ignoring API consistency.

Example of inconsistent APIs:

```
/users
```

```
/getProducts
```

```
/customer-list
```



Mixed naming conventions confuse API consumers.



7. PRODUCTION INSIGHT

Many API problems come from:

- ✗ Poor naming
- ✗ Inconsistent responses
- ✗ Missing versioning
- ✗ Lack of pagination rather than coding issues.



Good design saves more time than perfect code.



8. PRO TIP

Design APIs as if another team will use them.

Because eventually they probably will.



9. INTERVIEW INSIGHT

Q. Why should APIs use nouns instead of verbs?
Because HTTP methods (GET, POST, PUT, DELETE) already represent actions. The resource should be represented by the URL.



```
GET /GetUsers
```



```
GET /users
```



10. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Better readability
- ✓ Easier maintenance
- ✓ Consistent design
- ✓ Better developer experience

VS

DRAWBACKS

- ✗ Requires upfront planning
- ✗ Harder to change later
- ✗ Inconsistent design leads to issues



11. KEY TAKEAWAY

- ✓ REST best practices help create APIs that are predictable, maintainable, and developer-friendly.
- ✓ Consistency is often more important than perfection.



NEXT UP (DAY 71)

Pagination in APIs - Handling large datasets efficiently.



SCENARIO FOR YOU

If your API serves millions of records, would you return everything in one response or design the API for efficient pagination?



REMEMBER

- ✓ Keep it RESTful
- ✓ Keep it consistent
- ✓ Keep it simple





PAGINATION IN APIS

Handling large datasets efficiently



1. REAL SCENARIO

Your database contains:

- 1 Million Customers
- 5 Million Orders
- 10 Million Products

A client requests:

```
GET /products
```

Should your API return all records at once?

No!



That's when
Pagination becomes essential.



2. WHAT IS PAGINATION?

Pagination is the process of splitting large datasets into smaller chunks (pages).

Instead of returning:

100,000 records

Return:

20 records per page

This improves performance, scalability, and user experience.



3. REQUEST → API → RESPONSE FLOW



Why Pagination Matters

Without Pagination

- Slow responses
- High memory usage
- Network overhead
- Poor user experience

With Pagination

- Faster responses
- Lower memory usage
- Better scalability
- Better user experience

4. EXAMPLE REQUEST & RESPONSE

API Request:

```
GET /products?page=1&pageSize=20
```

API Response (Example):

```
{
  "page": 1,
  "pageSize": 20,
  "totalRecords": 1000,
  "totalPages": 50,
  "data": [
    { "id": 1, "name": "Product A" },
    { "id": 2, "name": "Product B" },
    // ... up to 20 records
  ]
}
```

Response Contains
Important Metadata

- Current page
- Page size
- Total records
- Total pages
- Data



5. TYPES OF PAGINATION

A. OFFSET PAGINATION

Uses page number and page size.

```
GET /products?page=2&pageSize=20
```

- Simple to implement
- Widely used
- Good for small to medium datasets



B. CURSOR PAGINATION

Uses a cursor/unique identifier.

```
GET /products?after=ABC123
```

- Better for very large datasets
- Efficient for real-time data
- Used in social media feeds, chat apps, activity timelines



6. COMMON PITFALL

Allowing clients to request huge page sizes.

```
GET /products?page=1&pageSize=100000
```

Result:

- Memory pressure
- Slow database queries
- API performance issues

Always enforce limits on page size!



7. PRODUCTION INSIGHT

Many production APIs set limits like:

- Default Page Size = 20
- Maximum Page Size = 100

This prevents abuse and protects system performance.



8. PRO TIP

Always return pagination metadata. It makes your API easier to consume.

- Current page
- Page size
- Total pages
- Total records
- Data



9. INTERVIEW INSIGHT

Q. Why is Cursor Pagination often faster than Offset Pagination?

Because databases don't need to skip a large number of records before fetching results.

OFFSET PAGINATION

Skip 50,000 records



Then fetch next 20

CURSOR PAGINATION

Start from cursor position



Fetch next 20

This becomes important for very large datasets.



10. BENEFITS vs DRAWBACKS

BENEFITS

- Better performance
- Better scalability
- Lower resource usage
- Improved user experience



DRAWBACKS (No Pagination)

- Large responses
- Slow APIs
- Higher infrastructure costs
- Poor scalability



VS



11. KEY TAKEAWAY

- Pagination improves API performance, scalability, and usability.
- Every API handling large datasets should implement a pagination strategy.



NEXT UP (DAY 72)

Filtering & Sorting in APIs – helping users find exactly what they need.



SCENARIO FOR YOU

If your products table grows from 1,000 records to 10 million records, would your API still perform well without pagination?



REMEMBER

- Use pagination by default
- Set sensible page size limits
- Return useful metadata
- Test with large datasets





FILTERING & SORTING IN APIs

Helping users find exactly what they need



1. REAL SCENARIO

Your products database contains

500,000+ Products

A client wants:

- Only Electronics
- Price less than ₹50,000
- Sorted by Rating

Should they download all products and filter them locally? 🤔



Definitely not!
That's where Filtering & Sorting become essential.



2. WHAT IS FILTERING?

Filtering allows clients to request only the data they need.

```
GET /products?category=Electronics
```

Instead of:

```
GET /products
```

and receiving everything.



3. WHAT IS SORTING?

Sorting allows clients to control result order.

Examples:

```
GET /products?sortBy=price
```

```
GET /products?sortBy=rating
```

4. REQUEST → FILTER → SORT → RESPONSE FLOW



This order is important for better performance!

5. EXAMPLE REQUEST & RESPONSE

API Request:

```
GET /products?
category=Electronics
&maxPrice=50000
&sortBy=rating
&page=1&pageSize=20
```

API Response (Example)

```
{
  "page": 1,
  "pageSize": 20,
  "totalRecords": 120,
  "totalPages": 6,
  "data": [
    {
      "id": 100,
      "name": "Laptop",
      "price": 45000,
      "rating": 4.5
    },
    // ... more items
  ]
}
```

Response Contains

- Filtered data
- Sorted data
- Pagination info
- Only what client actually needs



6. WHY FILTERING MATTERS

Without Filtering

- Large payloads
- Slow responses
- More database work
- Poor user experience

With Filtering

- Faster queries
- Better user experience
- Reduced network traffic
- Lower server load

7. COMMON SORTING OPTIONS

- Price Ascending
sortBy=price&sortOrder=asc
- Price Descending
sortBy=price&sortOrder=desc
- Rating
sortBy=rating&sortOrder=desc
- Created Date
sortBy=createdAtDate&sortOrder=desc
- Name (A to Z)
sortBy=name&sortOrder=asc

8. COMMON PITFALL

Allowing unrestricted sorting on every column.

```
GET /products?sortBy=anyColumn
```

This can:

- Hurt performance
- Create inefficient queries
- Expose sensitive columns

Always whitelist allowed fields for sorting and filtering!



9. PRODUCTION INSIGHT

Most production APIs combine:



+



+



Together:

Because returning filtered data is good.
Returning filtered + paginated data is even better.

10. PRO TIP

Keep parameter names simple and consistent.

```
?category=Electronics
&maxPrice=50000
&sortBy=rating
&page=1
&pageSize=20
```

- Predictable APIs are easier to use and integrate.

11. INTERVIEW INSIGHT

Q. Why should filtering happen in the database instead of the application?

- Because databases are optimized to filter large datasets efficiently.



Fetching everything and filtering in memory wastes resources, increases response time, and does not scale.

12. BENEFITS vs DRAWBACKS

With Filtering & Sorting

- Better performance
- Better scalability
- Better user experience
- Smaller payloads
- Lower bandwidth usage

VS

Without Filtering & Sorting

- Large payloads
- Slower APIs
- Poor user experience
- Increased server load
- Higher infrastructure cost



13. KEY TAKEAWAY

- Filtering and Sorting improve API usability, performance, and scalability.
- They allow clients to get exactly what they need.
- Combined with Pagination, they form the foundation of efficient data retrieval APIs.



NEXT UP (DAY 73)

Idempotency in APIs – preventing duplicate operations and improving reliability



SCENARIO FOR YOU

If your products table contains 10 million records, would you rather return everything or let clients request exactly what they need?



REMEMBER

- Filter early
- Sort efficiently
- Paginate always
- Validate parameters





IDEMPOTENCY IN APIs

Preventing duplicate operations in distributed systems.



1. REAL PAYMENT SCENARIO

A user clicks:

Pay Now

The payment request is sent.
But due to a network issue,
the user doesn't receive a response.
What do they do?

Click the button again.

Now your API receives
the same request twice.

Could the customer be
charged twice?

That's where
Idempotency becomes critical.



2. WHAT IS IDEMPOTENCY?

An operation is idempotent if performing it
multiple times produces the same result as
performing it once.

Example:

GET /products/1

Calling it:

1 time

or

100 times

Returns the same resource.



3. REAL PAYMENT FLOW



Client Sends Payment



Network Failure



Client Retries Request



API Detects Duplicate



Returns Existing Result



4. WHY IDEMPOTENCY MATTERS

- Prevents duplicate payments
- Prevents duplicate orders
- Handles retries safely
- Improves reliability
- Supports distributed systems



5. IDEMPOTENCY KEY EXAMPLE

Client sends:

POST /payments

Idempotency-Key:
abc123xyz

The API stores:

abc123xyz
→ Payment Success

If the same key arrives again:

- Return previous result
- Don't create another payment



6. COMMON PITFALL

Assuming retries never happen.
In reality:

- Mobile networks fail
- APIs timeout
- Clients retry requests

Without idempotency:

- Duplicate transactions
- Duplicate orders
- Data inconsistencies



7. PRODUCTION INSIGHT

Most payment providers use idempotency.

Examples include systems handling:



Payments



Order creation



Subscription
management

Because duplicate processing
can become expensive.



8. PRO TIP

Use idempotency for operations that
create or modify important business data.

Especially:

- Payments
- Orders
- Bookings
- Financial transactions



9. INTERVIEW INSIGHT

Q. Which HTTP methods are
naturally idempotent?

Idempotent

- GET
- PUT
- DELETE

Not Naturally Idempotent

- POST

That's why POST operations often
require idempotency keys.



10. BENEFITS vs DRAWBACKS

BENEFITS

- Prevents duplicates
- Safe retries
- Better reliability
- Improved user experience



VS

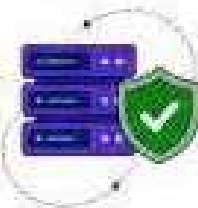
DRAWBACKS

- Additional storage
- Extra implementation complexity



11. KEY INSIGHT

Distributed systems assume failures will happen.
Good APIs are designed to recover safely
from these failures.



KEY TAKEAWAY

- Idempotency ensures repeated requests don't create duplicate outcomes.
- It's one of the most important concepts for building reliable APIs.



NEXT UP (DAY 74)

Caching in .NET -
reducing database load and
improving response times



SCENARIO FOR YOU

If a payment request times out
and the user clicks "Pay Now" again,
how would your API prevent a
duplicate charge?



REMEMBER

- Expect failures
- Design for retries
- Make operations idempotent



- Caching stores frequently used data so we don't hit the database every time

```

graph LR
    Request[Request] --> CheckCache[Check Cache]
    CheckCache --> Hit{Hit?}
    Hit -- YES (Cache Hit) --> ReturnData[Return Data from Cache]
    Hit -- NO (Cache Miss) --> FetchDB[Fetch from DB & Store in Cache]
  
```

The flowchart illustrates the logic for handling a request. It starts with a 'Request' (represented by a person icon), which leads to 'Check Cache' (represented by a server icon). From 'Check Cache', the flow goes to a decision point 'Hit?' (represented by a question mark icon). If the answer is 'YES (Cache Hit)', the flow proceeds to 'Return Data from Cache' (represented by a lightning bolt icon). If the answer is 'NO (Cache Miss)', the flow proceeds to 'Fetch from DB & Store in Cache' (represented by a database icon).

For most production apps,
Distributed Cache is preferred

Choose the strategy based on how often your data changes.

Good caching can reduce database load by 50% - 90%

Extra monetary charge
State share risk
Insulation complexity
Additional infrastructure
One plant per host nation

- Cache the right data
- Expire at the right time
- Invalidate when needed



REDIS BASICS

Understanding one of the most popular distributed caching solutions



1. REAL SCENARIO

Your application is running on multiple servers.
Users are constantly requesting:

- Product catalogs
- User profiles
- Configuration data
- Frequently accessed reports

Can an in-memory cache on one server help all servers?



No.



That's where Redis becomes powerful.



2. WHAT IS REDIS?

Redis (Remote Dictionary Server) is an in-memory data store.

It is commonly used for:

- Distributed caching
- Session storage
- Rate limiting
- Pub/Sub messaging
- Real-time applications



Redis keeps data in memory, making it extremely fast.

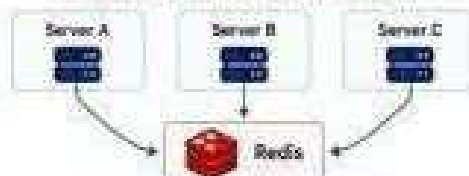


3. MULTI-SERVER vs REDIS ARCHITECTURE

Without Redis (In-Memory Cache on Each Server)



With Redis (Shared Distributed Cache)



All servers share the same cached data.



4. REDIS SETUP IN ASP.NET CORE

builder.Services

.AddStackExchangeRedisCache(opts =>

{
options.Configuration = "localhost:6379";
options.InstanceName = "MyApp";
});



Add Redis cache services in Program.cs



5. STORE & RETRIEVE DATA

STORE DATA

```
var productKey = $"product:{product.Id}";  
await cache.SetStringAsync(  
    productKey,  
    product.Name,  
    new DistributedCacheEntryOptions  
    {  
        AbsoluteExpirationRelativeToNow =  
            TimeSpan.FromHours(18)  
    });
```

RETRIEVE DATA

```
var productKey = $"product:{productId}";  
if (productKey is null)  
{  
    var product = await client.GetAsync<Product>(productKey);  
}
```



6. COMMON PITFALL

Treating Redis as a permanent database.

Redis is commonly used for:

- Temporary data
- Cached data
- Fast-access data



Not as a replacement for your primary database.



7. PRODUCTION INSIGHT

Many high-traffic systems use Redis for:

- API response caching
- Session management
- Authentication tokens
- Rate limiting



Because milliseconds matter at scale.



8. PRO TIP

Always set expiration times.



Product Cache
Expires in 10 minutes

This helps prevent stale data.



9. INTERVIEW INSIGHT

Q. Why is Redis faster than traditional databases?



Because data is stored primarily in memory (RAM), avoiding slower disk reads for many operations.



10. BENEFITS vs DRAWBACKS

BENEFITS

- Very fast
- Shared cache across servers
- Scalable
- Reduces database load

VS

DRAWBACKS

- Additional infrastructure
- Cache invalidation challenges
- Memory costs



11. KEY INSIGHT

Redis is not just a cache.
It's often a critical component for building scalable systems.



KEY TAKEAWAY



Redis enables distributed caching across multiple servers.
It helps applications scale while reducing pressure on databases.



NEXT UP (DAY 76)

Response Caching in ASP.NET Core
- Improving API performance with minimal code changes



SCENARIO FOR YOU

If your application runs on 10 servers,
where would you store shared
cached data so all servers
can access it?



REMEMBER

- Cache the right data
- Set expiration
- Keep data fresh





RESPONSE CACHING IN ASP.NET CORE

Improving API performance with minimal code changes



1. REAL SCENARIO

Your API endpoint:

`GET /products`

receives thousands of requests every minute.

But the product list only changes once every few hours.



Should your API generate the same response again and again?

Probably not.



That's where Response Caching becomes useful.



2. WHAT IS RESPONSE CACHING?

Response Caching stores the entire HTTP response and serves it directly for future requests.

Without Response Caching



With Response Caching



3. HOW RESPONSE CACHING WORKS



4. ENABLE RESPONSE CACING

```
builder.Services.AddResponseCaching();
app.UseResponseCaching();
```



Add these two lines and response caching is ready to use!



5. CACHE AN ENDPOINT

```
[ResponseCache(Duration = 60)]
public IActionResult GetProducts()
{
    return Ok(products);
}
```



This caches the response for 60 SECONDS



6. COMMON PITFALL

Caching data that changes frequently.

Example:

- Live stock prices
- Real-time dashboards
- Frequently changing inventory



This can lead to stale responses.



7. PRODUCTION INSIGHT

Response Caching works best for:

- Product catalogs
- Public content
- Configuration data
- Frequently requested read-only data



Not for highly dynamic information.



8. PRO TIP

Ask yourself:



"Would the response still be valid for the next minute?"

If yes, Response Caching may be a good candidate.



9. INTERVIEW INSIGHT

Q. What's the difference between Response Caching and Redis Caching?

Response Caching	Redis Caching
Caches the entire HTTP response.	Caches specific application data.
Easy to implement with minimal code.	More flexible and granular control.
Great for read-heavy and static data.	Useful for complex scenarios and distributed systems.



10. BENEFITS vs DRAWBACKS

BENEFITS

- Minimal implementation effort
- Easier APIs
- Reduced server load
- Better scalability

VS

DRAWBACKS

- Risk of stale data
- Limited control compared to Redis
- Not suitable for all endpoints



11. KEY INSIGHT

Sometimes the easiest performance optimization is simply avoiding unnecessary work.



KEY TAKEAWAY

- Response Caching improves API performance by serving cached HTTP responses instead of regenerating them.
- It's one of the simplest ways to improve performance for read-heavy APIs.



NEXT UP (DAY 77)

Distributed Caching — sharing cached data across multiple servers



SCENARIO FOR YOU

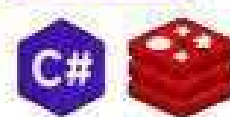
If your product catalog changes only twice a day but receives 50,000 requests per hour, would you generate the response every time or cache it?



REMEMBER

- Cache the right data
- Set appropriate expiration
- Invalidate when needed
- Measure performance





DISTRIBUTED CACHING IN .NET

Sharing cache across multiple servers



1. REAL-WORLD SCENARIO

Your application is running on three servers behind a load balancer.

- 1 User logs in.
- 2 First request hits Server A and user data is cached.
- 3 Next request goes to Server B...

The cache is missing! Database is queried again. Performance drops.

Distributed Caching solves this problem by sharing cache across all servers.

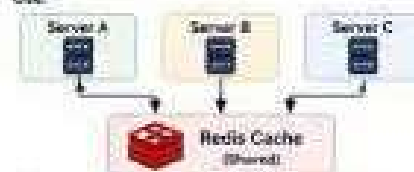
2. WHAT IS DISTRIBUTED CACHE?

Distributed Cache stores cached data in a shared external cache instead of each application's memory.

Instead of:



Use:



Now every server shares the same cached data.

3. WHY USE DISTRIBUTED CACHE?

- ✓ Shared across multiple servers
- ✓ Reduces database calls
- ✓ Faster response times
- ✓ Supports scalable applications

Popular Choice



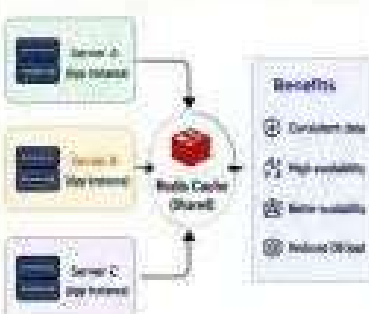
Redis

Fast, reliable and widely used in distributed systems.

4. IN-MEMORY CACHE vs DISTRIBUTED CACHE

IN-MEMORY CACHE	DISTRIBUTED CACHE
Stored inside one application instance	Stored externally (e.g., Redis)
Not shared across servers	Shared across all application instances
Lost if the application restarts	Persists independently of applications
Good for small applications	Best for scalable, cloud applications
Faster (no network call)	Slightly slower (network call)

5. ARCHITECTURE: MULTIPLE SERVERS → REDIS



6. ASP.NET CORE EXAMPLE

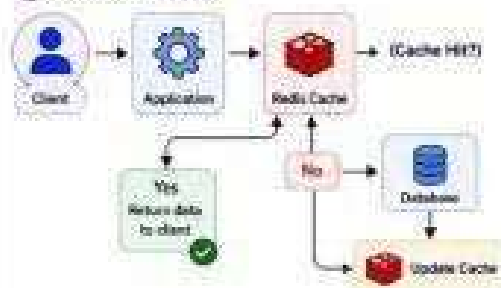
Register Redis Distributed Cache

```
services.AddDistributedCache<redisCache>(options =>
{
    options.Configuration = "localhost:6379";
    options.InstanceName = "MyApp";
});
```

Use DistributedCache in your services

```
public class CacheService
{
    private readonly IDistributedCache _cache;
    public CacheService(IDistributedCache cache)
    {
        _cache = cache;
    }
}
```

7. TYPICAL FLOW



8. COMMON PITFALL

Caching data without an expiration policy

Result:

- Stale data
- Incorrect responses
- Memory growth

Always define an appropriate expiration strategy.

9. PRODUCTION INSIGHT

Large-scale applications often use Redis for:

- Session storage
- Authentication tokens
- Product catalogs
- Frequently accessed lookup data

This reduces database load and improves scalability.

10. PRO TIP

Cache data that is:

- Frequently read
- Changes infrequently
- Expensive to compute or fetch

Avoid caching highly volatile or sensitive data unless there's a clear invalidation strategy.

11. INTERVIEW INSIGHT

Q: What is the difference between In-Memory Cache and Distributed Cache?

In-Memory Cache	Distributed Cache
Stored inside one application instance	Stored externally (e.g., Redis)
Not shared across servers	Shared across all instances
Lost if app restarts	Persists independently
Good for simple / single server applications	Best for cloud / multi-server environments

12. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Shared cache
- ✓ Better scalability
- ✓ Faster APIs
- ✓ Lower database load



DRAWBACKS

- Additional infrastructure
- Network dependency
- Cache invalidation complexity



13. KEY TAKEAWAY

- ✓ Distributed Cache helps multiple application instances share the same cached data.
- ✓ It's a key building block for scalable, high-performance cloud applications.



NEXT UP (DAY 78)

Background Jobs in .NET with Hangfire - running tasks outside the request pipeline.



SCENARIO FOR YOU

If your application runs on five servers behind a load balancer, would you cache data separately on each server or use a shared distributed cache?



REMEMBER

- Cache smartly
- Expire timely
- Invalidate wisely
- Scale effectively





BACKGROUND JOBS WITH HANGFIRE IN .NET

Running tasks outside the request lifecycle



HANGFIRE
BACKGROUND JOBS



1. REAL SCENARIO

A user completes a purchase.
Your API needs to:

- ✓ Save the order
- ✓ Send a confirmation email
- ✓ Generate an invoice PDF
- ✓ Notify the warehouse



Should the user wait until all these tasks finish?

Probably not.



That's where
Background Jobs
become useful.



2. WHAT ARE BACKGROUND JOBS?

Background jobs allow long-running or non-critical tasks to execute after the API responds.

WITHOUT BACKGROUND JOBS



WITH BACKGROUND JOBS



</> 3. HANGFIRE EXAMPLE

Enqueue a background job easily.

```
BackgroundJob.Enqueue(() =>
    emailService.SendConfirmation(orderId));
```

- ✓ Hangfire stores the job in the database.
- ✓ A background server picks it up and executes it.
- ✓ Works even if the application restarts.



Hangfire Dashboard

- View jobs
- in real-time
- Retry / Delete jobs
- Monitor failures
- See history & stats

4. TYPES OF JOBS IN HANGFIRE



1 FIRE-AND-FORGET



Runs once
immediately.

```
BackgroundJob.Enqueue(() =>
    SendEmail());
```



2 DELAYED JOB



Runs once
after a
specified delay.

```
BackgroundJob.Schedule(() =>
    SendEmail(), TimeSpan.FromMinutes(10));
```



3 RECURRING JOB



Runs on a
schedule
(daily, hourly,
weekly).

```
RecurringJob.AddOrUpdate("daily-report",
    () => GenerateReport(), Cron.Daily);
```



4 CONTINUATION JOB



Runs after
another job
completes.

```
BackgroundJob.ContinueJobWith(jobId,
    () => NextStep());
```



5. COMMON PITFALL

Moving critical business logic to a background job.

Example: Saving the order in a background job.



If the job fails, the order may
never be created.

- ✓ Use background jobs only for work that can happen after the main transaction succeeds. ✓



6. PRODUCTION INSIGHT

Many production systems use background jobs for:



Email
notifications



Report
generation



Data
synchronization



Cache
refresh



File
processing



Scheduled
maintenance

★ This keeps APIs responsive even under heavy load.



7. PRO TIP

Design background jobs to be
idempotent.

If a job runs twice because of a retry,
it shouldn't create duplicate data or
unexpected side effects.

- ✓ Always make jobs safe to run multiple times. ✓



8. INTERVIEW INSIGHT

Q. Why use Hangfire instead of writing your own Task.Run()?

Hangfire provides:

- ✓ Persistent storage
- ✓ Automatic retries
- ✓ Scheduling (delayed & recurring)
- ✓ Monitoring dashboard
- ✓ Reliable execution even after application restarts

VS

Task.Run()

- ✗ Lost if app restarts
- ✗ No built-in retries
- ✗ No scheduling
- ✗ No monitoring
- ✗ Not reliable for production



9. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Faster API response
- ✓ Better user experience
- ✓ Reliable execution
- ✓ Automatic retry support
- ✓ Easy scheduling

VS

DRAWBACKS

- ✗ Additional infrastructure (database)
- ✗ Job monitoring required
- ✗ Eventual consistency for background tasks



10. KEY TAKEAWAY



Background jobs improve API responsiveness by offloading long-running tasks.



Tools like Hangfire make scheduling, retries, and monitoring much easier than building your own solution.



NEXT UP (DAY 79)

Advanced Rate Limiting –
protecting APIs under
heavy traffic



SCENARIO FOR YOU

After a customer places an order, would you make their visit while an email and invoice are generated, or return the response immediately and process these tasks in the background?



REMEMBER

- Do the critical work in the request.
- Do the heavy work in the background.
- Keep users happy. Keep systems fast.





ADVANCED RATE LIMITING IN ASP.NET CORE

Protecting APIs under heavy traffic



1. REAL SCENARIO

Your public API suddenly receives

50,000
requests per minute!

Traffic includes:

- Real users
- Bots & scripts
- Accidental repeated requests



How do you protect your server
without blocking legitimate users?
Rate Limiting is the answer!



2. WHAT IS RATE LIMITING?

Rate Limiting controls how many requests
a client can make within a specific time
period.

WITHOUT RATE LIMITING



WITH RATE LIMITING



3. WHY RATE LIMITING MATTERS

- Prevents abuse
- Protects server resources
- Reduces DDoS impact
- Ensures fair usage
- Improves system stability



A small limit today can prevent a
major outage tomorrow.



4. COMMON ALGORITHMS

A. FIXED WINDOW

Example: 100 requests / minute



- Simple to implement
- Can allow bursts at window boundaries

B. SLIDING WINDOW

Tracks requests continuously
over time.



- Smoother
- More accurate

C. TOKEN BUCKET

Tokens are added over time.
Each request consumes one token.



- Best for APIs with occasional traffic bursts



5. ASP.NET CORE EXAMPLE

Configure Fixed Window Rate Limiting

```

builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("api",
        opt =>
        {
            opt.PermitLimit = 100; // max 100 requests
            opt.Window = TimeSpan.FromMinutes(1); // per 1 minute
            opt.QueueLimit = 10; // optional queue
        });
});
app.UseRateLimiter();
  
```

- This allows 100 requests per minute per client.
Extra requests will receive 429 status.



6. COMMON PITFALL

Applying the same limits to every endpoint.

EXAMPLE (NOT RECOMMENDED)

- /login 100 requests/min
- /products/search 100 requests/min

Different endpoints have different
importance and usage patterns.
Use different policies for different needs.



7. PRODUCTION INSIGHT

Use different limits for different users & services.

- Anonymous Users (Lower limits)
- Authenticated Users (Higher limits)
- Premium Customers (Higher limits)
- Internal Services (Custom policies)

This provides flexibility and protects
critical endpoints.



8. PRO TIP

Always return a proper 429 Too Many Requests
response with helpful headers.

HTTP/1.1 429 Too Many Requests

Retry-after: 60

Retry-After: 60

RateLimit-Remaining: 0

RateLimit-Reset: 60

"Retry-After: Too many requests. Please try again later."

- Retry-After: when to try again
- RateLimit-Limit: max requests allowed
- RateLimit-Remaining: remaining requests
- RateLimit-Reset: when the limit resets



9. INTERVIEW INSIGHT

Q. Difference between Fixed Window and Sliding Window?

FIXED WINDOW

- Simple to implement
- Counters reset at fixed intervals
- Can allow sudden traffic spikes at boundaries

VS

SLIDING WINDOW

- Tracks requests continuously
- More accurate
- Better traffic distribution
- Slightly more complex



10. BENEFITS vs DRAWBACKS

BENEFITS

- Protects infrastructure
- Prevents abuse & attacks
- Fair resource usage
- Better reliability & availability

VS

DRAWBACKS

- Requires careful tuning
- Poor limits may affect legitimate users
- Complex policies can be hard to manage



11. KEY TAKEAWAY

Rate Limiting isn't about
blocking users.

It's about ensuring your API
remains available, secure,
and responsive for everyone.



NEXT UP (DAY 80)

SQL Privileges Revisited -
applying SQL in real-world
backend applications.



SCENARIO FOR YOU

If your login endpoint and product catalog
endpoint receive the same traffic limit,
could that negatively affect the
user experience?



REMEMBER

- Understand your traffic patterns
- Choose the right algorithm
- Use proper limits per endpoint
- Monitor, test, and adjust continuously





SOLID PRINCIPLES REVISITED

Applying SOLID in real-world backend applications



Better Code. Better Design. Better Systems.



1. REAL SCENARIO

You are building an Order Management API. As the system grows, you need to:

- ➡ Add new payment gateways
- ➡ Support multiple notification channels
- ➡ Handle different order types
- ➡ Keep the code maintainable



SOLID principles help you build systems that are easy to extend and hard to break.



2. WHAT IS SOLID?

SOLID is a set of 5 design principles by Robert C. Martin (Uncle Bob) that make software more maintainable, scalable and easier to understand.



Good design today = Flexible systems tomorrow.



3. SOLID OVERVIEW

SOLID is an acronym for five design principles.

- S** Single Responsibility Principle: Do one thing, do it well.
- O** Open/Closed Principle: Open for extension, closed for modification.
- L** Liskov Substitution Principle: Subtypes must be substitutable.
- I** Interface Segregation Principle: Many specific interfaces are better than one general interface.
- D** Dependency Inversion Principle: Depend on abstractions, not on concretions.

★ 4. THE 5 SOLID PRINCIPLES WITH EXAMPLES



SINGLE RESPONSIBILITY PRINCIPLE

A class should have only one reason to change.

❌ BAD EXAMPLE

OrderService
initiate order + mail + pdf generation.

✅ GOOD EXAMPLE

OrderService → handles orders
OrderService → sends email
PdfService → generates PDFs

- 1 Separation makes code easier to maintain.



OPEN/CLOSED PRINCIPLE

Software entities should be open for extension but closed for modification.

❌ BAD EXAMPLE

Adding a new payment method requires changing existing code.

✅ GOOD EXAMPLE

Use abstractions, and add new implementations.

```
PaymentGateway
CreditCardGateway
PayPalGateway
UPIGateway
```

- 1 Extends behavior without changing existing code.



LISKOV SUBSTITUTION PRINCIPLE

Subtypes must be substitutable for their base types.

❌ BAD EXAMPLE

Derived class changes the behavior unexpectedly.

✅ GOOD EXAMPLE

Derived classes should honor the base class contract.

```
class Base {
    public virtual void fly();
}
// derive from fly - ok
// derive from fly - mistake
class Derived {
    public void fly() { ... }
}
```

- 1 Trust is the foundation of substitution.



INTERFACE SEGREGATION PRINCIPLE

Clients should not be forced to depend on interfaces they do not use.

❌ BAD EXAMPLE

Dependency with 20 methods - clients depend on all.

✅ GOOD EXAMPLE

Create small, focused interfaces.

```
interface OrderCut {
    // ...
}
interface OrderRepository {
    GetOrder()
}
```

- 1 Smaller interfaces = Happier clients.



DEPENDENCY INVERSION PRINCIPLE

Depend on abstractions, not on concretions.

❌ BAD EXAMPLE

High level modules depend on concrete classes.

✅ GOOD EXAMPLE

Depend on interfaces and inject implementations.

```
class OrderService {
    private readonly IPaymentGateway _gateway;
}
// Inject any implementation
```

- 1 Decoupled code is easy to test and extend.

</> 5. REAL-WORLD EXAMPLE

Scenario: Adding a new Payment Gateway



6. COMMON PITFALLS

- ➡ Ignoring SOLID for the sake of speed.
- ➡ Creating God classes that do everything.
- ➡ Using inheritance when composition is better.
- ➡ Having large interfaces with unrelated methods.
- ➡ Tight coupling between layers.



SOLID is not about more code. It's about the right design.



7. PRODUCTION INSIGHT

SOLID helps when:

- ➡ Requirements keep changing
- ➡ New features are added frequently
- ➡ Multiple teams work on the same codebase
- ➡ You need long-term maintainability
- ➡ You want clean, testable code



SOLID reduces the cost of change as your system grows.



8. PRO TIP

Apply SOLID where it matters most. Don't over-engineer simple solutions.

Start small. Refactor as the system grows.

- ➡ TDD + SOLID = Perfect combination.



9. INTERVIEW INSIGHT

Q. Why is SOLID important?

- ➡ It improves code maintainability
- ➡ It makes the system easier to extend
- ➡ It reduces coupling between components
- ➡ It improves testability
- ➡ It helps teams collaborate effectively



10. BENEFITS vs DRAWBACKS

BENEFITS	DRAWBACKS
➡ Maintainable code ➡ Scalability ➡ Flexibility ➡ Reusability ➡ Easier testing	➡ More upfront design effort ➡ Slight learning curve ➡ Can be overused (Over-engineering)



11. KEY TAKEAWAY

SOLID principles are not just theory. They are practical guidelines that help you build flexible, maintainable and scalable backend systems.



Write code that is easy to change today, so your future self (and your team) will thank you.



NEXT UP (DAY 81)

Domain Driven Design (DDD) - Building complex systems with a clear domain focus.



SCENARIO FOR YOU

If you add a new payment gateway monthly, would you rather change existing code every time or add it without touching existing code?



REMEMBER

- ➡ SOLID is a mindset.
- ➡ Design for change.
- ➡ Keep it simple.
- ➡ Keep it clean.





DOMAIN-DRIVEN DESIGN (DDD) BASICS

Building Software Around the Business Domain



1. REAL SCENARIO

You're building an E-commerce application. The business introduces new rules:

- ✓ VIP customers get special discounts
- ✓ Orders above ₹10,000 require manager approval
- ✓ Some products cannot be shipped internationally



Where should these business rules live?

- ✗ Controllers?
- ✗ Database?
- ✗ Random utility classes?



This is where Domain-Driven Design (DDD) becomes valuable.



2. WHAT IS DDD?

Domain-Driven Design is an approach to software development that focuses on the business domain first.

Instead of designing around:

- ✗ Database tables
- ✗ Frameworks
- ✗ APIs

You design around:

- ✓ Business concepts
- ✓ Business rules
- ✓ Real-world processes

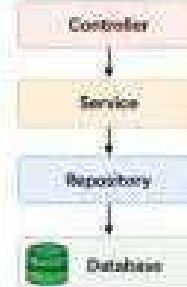


The goal is to build software that matches how the business actually works.



3. TRADITIONAL vs DDD

TRADITIONAL DESIGN



- ✗ Business rules are often scattered across different layers.

DOMAIN-DRIVEN DESIGN



- ✓ The Domain layer contains the core business logic.



4. CORE BUILDING BLOCKS OF DDD

1 ENTITY

Has a unique identity that does not change.

Examples:

- Customer
- Order
- Product

Different from Value Objects.

2 VALUE OBJECT

Has no identity. Defined only by its values.

Examples:

- Address
- Money
- Email

Immutable and replaceable.

3 AGGREGATE

A group of related entities treated as a single unit.

Example: Order Aggregate



Changes are made through the Aggregate Root.

4 REPOSITORY

Provides access to aggregates without exposing database details.

Example:

```
OrderRepository {
    Order GetById(int id);
    void Save(Order order);
}
```

Works with Aggregates.

5 DOMAIN SERVICE

Contains domain logic that doesn't naturally belong to an entity or value object.

Example:

Discount Calculation
Approval Workflow
Notification Logic

Stateless and focused on domain tasks.



5. COMMON PITFALL

Putting all business rules inside:

- ✗ Controllers
- ✗ Services
- ✗ Stored Procedures
- ✗ Utility Classes

DDD encourages keeping business rules inside the Domain Model, making them easier to maintain and test.



6. PRODUCTION INSIGHT

Large enterprise applications often adopt DDD because:

- ✓ Business rules evolve frequently
- ✓ Teams grow over time
- ✓ Complex domains become easier to manage
- ✓ New features are added regularly



However, for very small CRUD applications, DDD may introduce unnecessary complexity.



7. PRO TIP

Don't force DDD into every project.

Use it when your business domain is complex and the rules change often.

Start simple. Evolve toward DDD as needed.



8. INTERVIEW INSIGHT

Q: What is the primary goal of Domain-Driven Design?

A: To model software around the business domain so that business rules are clearly represented, easier to understand, and simpler to maintain as requirements evolve.



9. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Better business alignment
- ✓ Easier maintenance
- ✓ Highly testable
- ✓ Scales well for large systems
- ✓ Clear separation of responsibilities



VS

DRAWBACKS

- ✗ Higher learning curve
- ✗ More initial design effort
- ✗ Can be excessive for small projects



10. KEY INSIGHT

DDD isn't about adding more layers.

It's about making the BUSINESS DOMAIN the center of your application so that the software evolves naturally with business needs.



The most maintainable applications aren't built around databases—they're built around the business they serve.



11. TAKEAWAY

Domain-Driven Design helps you build software that reflects real-world business logic. It improves maintainability, collaboration, and long-term scalability when used wisely.



NEXT UP (DAY 82)

Event-Driven Architecture - Building loosely coupled, scalable systems



SCENARIO FOR YOU

If your business introduces a new shipment policy tomorrow, would you rather update business rules in one central domain model or search through multiple controllers and services?



REMEMBER

- ✓ Understand the domain
- ✓ Use the right building blocks
- ✓ Keep business rules in the domain
- ✓ Evolve with the business



OUTBOX PATTERN IN MICROSERVICES



EVENT

Keeping your database and message broker in sync



1. REAL SCENARIO

A customer places an order.

- Order is saved in the database.
- Then the application tries to publish OrderCreated event.

But suddenly...

- Message broker is unavailable.



Order exists in DB, but other services never receive the event.

This creates inconsistent systems.



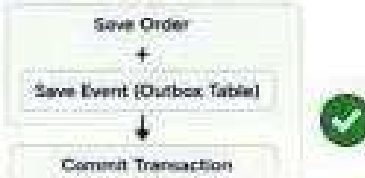
The Outbox Pattern helps keep everything in sync.

2. WHAT IS THE OUTBOX PATTERN?

Instead of:

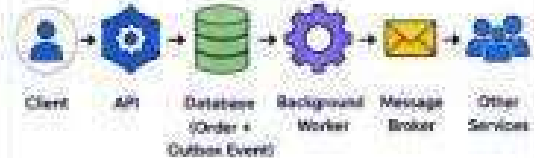


The application does this:



Both happen in the same database transaction.

3. HOW IT WORKS



If the broker is temporarily unavailable, the event safely remains in the Outbox table until it can be published.

4. EXAMPLE OUTBOX RECORD

EventId	EventType	Payload	Status	CreatedAt
101	OrderCreated	{ "orderId": 123, "amount": 1599 }	Pending	2024-06-30 18:30:25



A background service later reads pending events and publishes them to the message broker.

5. WHY TEAMS USE THE OUTBOX PATTERN



Presents fault events



Improves reliability



Ensures eventual consistency



Supports retry mechanisms

6. COMMON PITFALL

Publishing events before the database transaction completes.



- Other services think the order exists.
- But the database rolled it back.
- This leads to inconsistent data.

7. PRODUCTION INSIGHT

Many distributed systems use:

- Outbox Pattern
- Background workers
- Retry policies



to reliably deliver messages without requiring distributed transactions.



8. PRO TIP

The Outbox Pattern does NOT guarantee instant delivery.



It guarantees reliable delivery.

That's usually more important in distributed systems.

9. INTERVIEW INSIGHT

Q. Why not simply save the data and publish the message immediately?

Answer:

Because if one succeeds and the other fails, your system becomes inconsistent. The Outbox Pattern ensures the event isn't lost.



10. BENEFITS vs TRADE-OFFS

BENEFITS

- Reliable event publishing
- Better fault tolerance
- Eventual consistency
- Easier retries

VS

TRADE-OFFS

- Additional Outbox table
- Background processing required
- Slight increase in implementation complexity



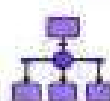
KEY INSIGHT

Distributed systems don't fail because code is wrong. They often fail because communication between services is unreliable. Patterns like the Outbox Pattern are designed to handle those failures gracefully.



NEXT UP (DAY 84)

Saga Pattern - Coordinating transactions across multiple microservices



SCENARIO FOR YOU

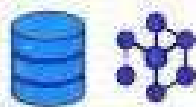
If saving an order succeeds but publishing the OrderCreated event fails, how would the inventory or Payment service know that a new order exists?



REMEMBER

- Store the event in Outbox table
- Publish using background worker
- Retry until success
- Ensure eventual consistency

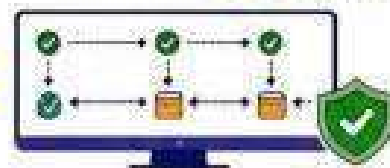




MICROSERVICES

SAGA PATTERN IN MICROSERVICES

Managing transactions across multiple services



1. REAL SCENARIO

A customer places an order. Your system needs to:

- ✓ Create the Order
- ✓ Reserve Inventory
- ✓ Process Payment
- ✓ Arrange Shipping

What if the payment fails after inventory is reserved?

- ✗ Inventory stays reserved
- ✗ Order remains pending
- ✗ Services become inconsistent



The Saga Pattern helps maintain consistency.

2. WHAT IS THE SAGA PATTERN?

A Saga is a sequence of local transactions across multiple microservices.

- ✓ Each service completes its own transaction
- ✓ Publishes an event
- ✓ Triggers the next service



If one step fails, compensating actions undo the previous successful steps.

KEY IDEA



3. SAGA SUCCESS FLOW



✓ All steps succeed → Order is completed.

4. SAGA FAILURE & COMPENSATION FLOW



Compensating actions restore the system to a consistent state.

5. WHY TEAMS USE SAGA PATTERN



Maintains consistency across services



Eliminates distributed transactions



Improves scalability



Works well in event-driven architectures

6. COMMON PITFALL

Treating compensation as a simple database rollback.

DO NOT DO THIS



Database Rollback



DO THIS INSTEAD

- Cancel Order
- Refund Payment
- Release Inventory



Compensating action is a new business operation, not a rollback.

7. PRODUCTION INSIGHT

Modern microservices rarely use distributed transactions because they:

- ✗ Reduce scalability
- ✗ Increase coupling

Instead, they rely on:

- ✓ Events
- ✓ Sagas
- ✓ Retry policies
- ✓ Idempotent operations



8. PRO TIP

Design every Saga step with failure in mind.

Ask yourself:

"If this step succeeds but the next one fails, how do I safely recover?"

9. INTERVIEW INSIGHT

Q. What's the difference between the Outbox Pattern and the Saga Pattern?

OUTBOX PATTERN

Ensures reliable event publishing from a single service.



VS

SAGA PATTERN

Coordinates business transactions across multiple services.



They solve different problems but are often used together.

10. BENEFITS vs TRADE-OFFS

BENEFITS

- ✓ Better scalability
- ✓ Fault tolerance
- ✓ Eventual consistency
- ✓ Independent services

VS

TRADE-OFFS

- ✗ More complex workflows
- ✗ Compensation logic required
- ✗ Harder debugging



KEY TAKEAWAY

The Saga Pattern helps coordinate long-running business transactions across multiple microservices using events and compensating actions instead of distributed transactions.



NEXT UP (DAY 85)

Common Microservices Pitfalls - mistakes teams make and how to avoid them.



SCENARIO FOR YOU

If payment succeeds but shipping fails, would you keep the order in-flight, or design a compensation workflow to maintain business consistency?



REMEMBER

- ✓ Think in events, not transactions.
- ✓ Design for failure, not success.
- ✓ Compensation is not rollback.
- ✓ Consistency is achieved, not assumed.



COMMON MICROSERVICES PITFALLS

When microservices create more problems than they solve



1. REAL SCENARIO

A team starts a new project and splits it into multiple microservices.

- User Service
- Order Service
- Payment Service
- Inventory Service
- Notification Service



Everything looks modern... until deployment.

- Communication failures
- Distributed debugging
- Data consistency issues
- Slow deployments

What went wrong? They adopted microservices before they actually needed them.

2. WHAT ARE MICROSERVICES?

Microservices divide an application into smaller, independently deployable services.



Owns its business capability



Has its own database (ideally)



Can be deployed independently

“ Small services, loosely coupled, working together.”

3. TYPICAL ARCHITECTURE



Each service is independent and communicates over the network.

4. COMMON PITFALLS

1. SPLITTING TOO EARLY



Many projects don't need microservices.

A modular monolith is often easier to build, deploy, and maintain.

2. TIGHT COUPLING BETWEEN SERVICES



If every service depends on multiple others to complete one request:

- Failures cascade
- Deployments become risky

Services should communicate only when necessary.

3. DISTRIBUTED TRANSACTIONS



Updating multiple databases across services is difficult.

Instead of traditional transactions:

- Use eventual consistency
- Consider patterns like Saga or Outbox

4. POOR OBSERVABILITY



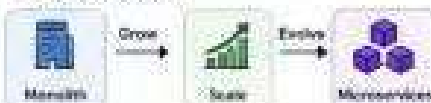
Without proper observability:

- Centralized logging
- Distributed tracing
- Metrics

Finding the root cause of failures becomes extremely difficult.

5. PRODUCTION INSIGHT

Many successful companies begin with a monolith and moved to microservices only after scaling requirements justified the added complexity.



Architecture should solve today's problems – not tomorrow's assumptions.

6. PRO TIP

“ Don't ask: “Can we build microservices?”

Ask: “Should we build microservices?”

Technology should follow business needs.



7. INTERVIEW INSIGHT

Q: When should you choose microservices over a monolith?

A good answer includes:

- Independent scaling requirements
- Large engineering teams
- Independent deployments
- Clear business boundaries

★ Not simply because microservices are popular.

8. BENEFITS vs CHALLENGES

BENEFITS

- Independent deployments
- Better scalability
- Team autonomy
- Technology flexibility



CHALLENGES

- Operational complexity
- Distributed debugging
- Network latency
- Data consistency
- Higher infrastructure costs



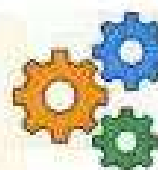
VS

9. KEY TAKEAWAY

“ Microservices are an architectural trade-off – not an upgrade.”

The best architecture is the one that fits your team's size, product maturity, and business goals.

Start simple. Evolve your architecture only when the business demands it.



NEXT UP (DAY 86)

Azure App Service – Hosting ASP.NET Core Applications in the Cloud.



SCENARIO FOR YOU

If you're building a startup MVP with a small team, would you choose a monolith or microservices? Why?



REMEMBER

- Choose the right architecture
- Measure and monitor everything
- Keep services independent
- Evolve continuously





1. REAL SCENARIO

You've built an ASP.NET Core Web API. It works perfectly on your local machine.



"Can we make it available to users worldwide?"

You need a reliable place to host your application.

That's where Azure App Service comes in!

2. WHAT IS AZURE APP SERVICE?

Azure App Service is a fully managed Platform as a Service (PaaS) for hosting web applications, REST APIs, and background services.



Microsoft manages:

- ✓ Servers
- ✓ OS updates
- ✓ Security patches
- ✓ Scaling infrastructure

So developers can focus on building applications instead of managing servers.

3. DEPLOYMENT FLOW



Your code goes live and is accessible to users around the world! 🌐

4. WHY TEAMS USE AZURE APP SERVICE

- 🚀 Quick deployments
- 🔒 Built-in HTTPS/SSL
- 📈 Auto-Scaling
- 🔄 Deployment Slots
- 🌐 High Availability
- 🔄 CI/CD Integration

5. COMMON DEPLOYMENT OPTIONS

- Visual Studio Publish
- GitHub Actions
- Azure DevOps Pipelines
- Azure CLI

Choose the method that best fits your team's workflow.

6. COMMON PITFALL

Keeping production secrets inside `appsettings.json`

```

appsettings.json
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=...;
    Password=MySecret123!"
  },
  "AppKeys": {
    "PaymentGateway": "abc123xyz"
  }
}
  
```

Instead use:

- Azure App Settings
- Azure Key Vault

This keeps sensitive information secure.

7. PRODUCTION INSIGHT

Many production applications use:
GitHub → CI/CD Pipeline → Azure App Service



Every code merge automatically deploys the latest version after passing validation. This reduces manual errors and improves release consistency.

8. PRO TIP

Use Deployment Slots.



1. Deploy new version to Staging slot
2. Test everything thoroughly
3. Swap with Production (zero/minimal downtime)

★ This makes releases safer and rollback easier!

9. INTERVIEW INSIGHT

Q: Why choose Azure App Service over managing your own Virtual Machine?

A good answer includes:

- ✓ No server management
- ✓ Built-in scaling
- ✓ Easier deployments
- ✓ Automatic patching
- ✓ Lower operational overhead

★ Focus on productivity, reliability, and reduced operations work.

10. BENEFITS vs CHALLENGES

BENEFITS

- ✓ Fully managed hosting
- ✓ Faster deployments
- ✓ Built-in monitoring (Application Insights)
- ✓ Auto-scaling
- ✓ Easy SSL/HTTPS support
- ✓ High availability



CHALLENGES

- ✗ Less OS-level control
- ✗ Platform limits for certain workloads
- ✗ Cost planning becomes important at scale
- ✗ Some custom configurations may not be possible



11. KEY TAKEAWAY

Cloud hosting isn't just about putting your application online.

It's about improving reliability, scalability, and deployment efficiency.

Azure App Service helps teams deploy, manage, and scale ASP.NET Core applications without worrying about server infrastructure.

Build your application. Let Azure handle the platform.



NEXT UP (DAY 87)

Azure Key Vault – Securing Secrets, Keys, and Certificates in Production



SCENARIO FOR YOU

If your API stores database passwords and API keys, would you keep them inside your project files or use a secure secret management service?



REMEMBER

- 1. Use App Settings or Key Vault
- 2. Keep secrets out of source code
- 3. Automate deployments
- 4. Monitor and scale proactively





1. REAL SCENARIO

Your ASP.NET Core API needs:

- SQL Database Connection String
- JWT Secret Key
- Storage Account Key
- Third-party API Keys

Where should you store them?

- appsettings.json
- Source code
- GitHub repository



Instead...
Azure Key Vault.

2. WHAT IS AZURE KEY VAULT?

Azure Key Vault is Microsoft's secure secret management service.

It helps store:

- Secrets
- Encryption Keys
- SSL Certificates



without exposing them in your application code.

3. HOW IT WORKS



The application requests secrets only when needed.

4. WHAT CAN YOU STORE?

- Database Connection Strings
- JWT Signing Keys
- API Keys
- Storage Keys
- Certificates
- Encryption Keys

5. WITHOUT KEY VAULT

```

appsettings.json
{
  "ConnectionString":
  "Server=tcp:myserver.database.windows.net;Database=Prod;
  User Id=admin;Password=12345";
  "JwtSecret": "MySuperSecretKey",
  "ApiKey": "abc123xyz"
}
  
```

If this file is committed accidentally...
Your secrets are exposed.

6. WITH KEY VAULT



Secrets stay outside your codebase.

7. COMMON PITFALL

Storing secrets in:

- Git repositories
- Configuration files
- Environment files shared publicly



Even private repositories can become public accidentally.

8. PRODUCTION INSIGHT



- No passwords are hardcoded.
- No credentials are stored in source code.
- Secrets are retrieved securely at runtime.
- This improves security and simplifies secret rotation.

Many enterprise applications use Managed Identity to access Key Vault – no secrets in code, no secrets in config.

9. PRO TIP

Rotate secrets regularly.



Even secure secrets shouldn't remain unchanged forever. Good security is an ongoing process.

10. INTERVIEW INSIGHT

Q: Why is Azure Key Vault preferred over storing secrets in appsettings.json?

A good answer includes:

- Centralized secret management
- Better security
- Secret rotation
- Access control with Azure AD
- Auditing and monitoring

It's not just about storing secrets, it's about managing them securely.

11. BENEFITS vs CHALLENGES

BENEFITS

- Strong security
- Centralized secret management
- Secret rotation
- Access auditing
- Reduced risk of leaks



VS

CHALLENGES

- Initial Azure configuration
- Azure permissions must be managed carefully
- Small learning curve for new teams



12. KEY TAKEAWAY

“Your application's source code should never contain secrets.”

Applications should request secrets securely – not own them permanently.

Azure Key Vault helps protect sensitive information by keeping secrets outside your application and providing secure, centralized access.

Build secure applications from day one – not after an incident.



NEXT UP (DAY 88)

Azure Storage – Blob, Queue, Table & File Storage Explained



SCENARIO FOR YOU

If your GitHub repository becomes public today, would any passwords or API keys be exposed?



REMEMBER

- Never commit secrets to code
- Use Key Vault for all sensitive data
- Use Managed Identity
- Rotate secrets regularly





AZURE STORAGE

BLOBS, FILES, QUEUES & TABLES



The right storage for the right data and workload



1. REAL SCENARIO

Your application needs to:

- ✓ Store profile pictures
- ✓ Upload invoices and PDFs
- ✓ Process background tasks
- ✓ Save large amounts of structured data



Should everything go into SQL Server?
Not always.



That's where Azure Storage comes in.



2. WHAT IS AZURE STORAGE?

Azure Storage is a scalable cloud storage service designed for different types of data.



Instead of one storage solution for everything, Azure provides specialized services for different needs.



Scalable



Durable



Cost-effective



Secure



Azure Integrated



3. TYPES OF AZURE STORAGE



BLOB STORAGE

Best for unstructured data like images, videos, documents, backups.
Examples: profile-photo.jpg, invoice.pdf, product-cms.mp4



AZURE FILES

Provides shared file storage accessible from multiple machines.
Use cases: Shared folders, legacy apps, lift-and-shift migrations



QUEUE STORAGE

Stores messages between applications.
Use cases: Background tasks, order processing, email sending, notifications

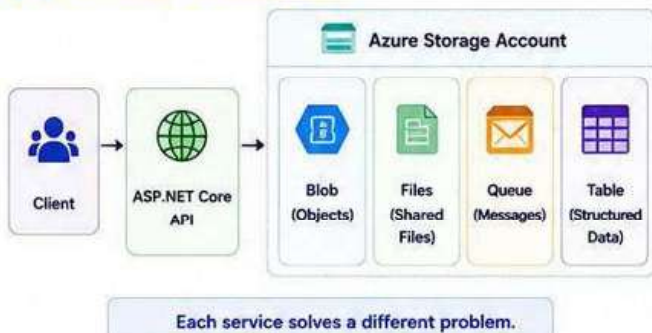


TABLE STORAGE

NoSQL key-value store for large volumes of structured, non-relational data.
Use cases: Logs, IoT data, metadata, high-scale applications



4. TYPICAL ARCHITECTURE



5. WHEN TO USE WHAT?

Service	Best For	Examples
Blob Storage	Unstructured data (Binary Large Objects)	Images, Videos, PDFs, Backups, Documents
Azure Files	Shared file storage	Shared folders, Config files, SMB file shares
Queue Storage	Asynchronous messaging	Background jobs, Email sending, Order processing
Table Storage	Structured, non-relational data at scale	Logs, IoT data, Metadata, User activity



6. COMMON PITFALL

Using SQL Server to store large files like images, videos and documents.

- ✗ Bigger database size
- ✗ Slower backups
- ✗ Increased storage costs
- ✗ Poor performance



Blob Storage is usually a better choice.



7. PRODUCTION INSIGHT

A common enterprise pattern:



Images / Videos
→ Blob Storage



Background Tasks
→ Queue Storage



Shared Documents
→ Azure Files



Large Structured Data
→ Table Storage



Choosing the right storage service improves performance, scalability and cost.



8. PRO TIP



Choose storage based on your data type, not just familiarity.

One size doesn't fit all!



9. INTERVIEW INSIGHT

Q. When would you use Blob Storage instead of SQL Server?

Answer:

When storing large binary files such as images, videos, PDFs or backups. SQL Server should primarily manage relational data, while Blob Storage is optimized for file storage.



10. BENEFITS vs DRAWBACKS

BENEFITS

- ✓ Highly scalable
- ✓ High availability and durability
- ✓ Pay-as-you-go pricing
- ✓ Multiple storage options
- ✓ Easy integration with Azure services
- ✓ Secure and compliant



DRAWBACKS

- ✗ Requires choosing the right storage type
- ✗ Additional learning compared to traditional databases
- ✗ Network latency for some operations



11. KEY TAKEAWAY

- ✓ Azure Storage provides specialized services for files, messages and structured data.
- ✓ Selecting the right storage type leads to better scalability, performance and cost optimization.
- ✓ Not all data belongs in a database. Modern cloud applications choose the right storage service based on the workload.



NEXT UP (DAY 89)

Azure Service Bus – Building reliable communication between distributed applications



SCENARIO FOR YOU

If users upload thousands of images every day, would you store those images inside SQL Server or use Blob Storage? Why?



REMEMBER

- ✓ Right data in the right storage
- ✓ Design for scale from day 1
- ✓ Cost, performance and security matter





AZURE SERVICE BUS IN .NET

Building reliable communication between applications



1. REAL SCENARIO

A customer places an order.

Your application needs to:

- Save the order
- Send a confirmation email
- Update inventory
- Generate an invoice
- Notify the shipping system

Should the Order API wait for all these tasks to finish before responding?

- ❌ No! That would make the API slow and tightly coupled.

Instead, let the Order API send a message and allow other services to process it asynchronously.

That's where Azure Service Bus comes in.

2. WHAT IS AZURE SERVICE BUS?

Azure Service Bus is a cloud messaging service that enables applications and microservices to communicate asynchronously.

Instead of calling another service directly:

DIRECT API CALLS (Tightly Coupled)



Use a message broker:

WITH AZURE SERVICE BUS (Loosely Coupled)



Each service processes messages independently.

3. WHY USE AZURE SERVICE BUS?

- Decouples applications
- Improves scalability
- Handles temporary failures
- Supports asynchronous processing
- Enables reliable communication

4. TYPICAL MESSAGE FLOW



5. QUEUES vs TOPICS



A. QUEUE

- One producer
- One consumer processes each message

Best for:

- Order processing
- Background jobs



Each message is consumed by only one receiver (First come, first served).

VS



B. TOPIC

- One producer
- Multiple subscribers receive the same message

Best for:

- Notifications
- Event-driven systems
- Microservices



Each message is broadcasted to all subscribers.

6. COMMON PITFALL

Treating Service Bus like a database.

Messages should be:

- Small
- Temporary
- Focused on communication



Not long-term storage.

“Service Bus is for moving messages, not storing data.”

7. PRODUCTION INSIGHT

Many enterprise systems use Azure Service Bus to:

- Process orders
- Send emails
- Integrate microservices
- Retry failed operations



This keeps applications responsive during traffic spikes.



8. PRO TIP

Design your message consumers to be idempotent.

If the same message is processed more than once, the outcome should remain correct.



Idempotent Consumer

9. INTERVIEW INSIGHT

Q. When should you use Azure Service Bus instead of direct API calls?

A. When services don't need an immediate response, or when you want loose coupling, better scalability, and reliable asynchronous communication.

10. BENEFITS vs DRAWBACKS



Loose Coupling



Better Scalability



Reliable Messaging



Fault Tolerance

VS



Increased Complexity



Requires Monitoring



Eventual Consistency



Drawbacks



11. KEY TAKEAWAY

- Azure Service Bus helps applications communicate reliably without being tightly connected.
- Asynchronous messaging improves scalability, resilience, and overall system performance.



NEXT UP (DAY 90)

Application Monitoring & Observability - how production systems stay healthy



SCENARIO FOR YOU

If your Cloud Service is temporarily unavailable, would you often fail the user's order immediately or queue the request and process it as soon as the service is available?



REMEMBER

- Decouple
- Queue
- Process
- Success





Keeping production systems healthy



1. REAL PRODUCTION SCENARIO

Your API is suddenly slow.
Users report timeouts.

But...

- ❌ CPU looks normal
- ❌ Database is running
- ❌ No deployment happened

How do you find the real problem?

You need **Visibility**.You need **Observability**.Monitoring tells you **WHAT** is happening.Observability helps you understand **WHY**.

2. MONITORING vs OBSERVABILITY

MONITORING

Answers:

"Is my application healthy?"

- 📊 CPU Usage
- 📊 Memory Usage
- 📊 Request Count
- 🕒 Response Time
- 🕒 Error Rate
- 📊 Availability

Focus: Detect problems

OBSERVABILITY

Answers:

"Why is my application unhealthy?"

- 📄 Logs
- 📊 Metrics
- 📊 Distributed Traces
- 📄 Events
- 📊 Business Context

Focus: Understand problems

VS

Monitoring detects the problem.
Observability helps you find the root cause.

3. THREE PILLARS OF OBSERVABILITY



LOGS

Capture discrete events.

Example:

```
User login failed for userId: 123
Reason: Invalid password
```



METRICS

Measure numeric values over time.

Examples:

- Response Time (ms)
- Requests per Second
- CPU / Memory Usage
- Error Rate (%)



TRACES

Track a request as it flows across multiple services.

Example:



Logs + Metrics + Traces = Full Observability

4. TYPICAL MONITORING ARCHITECTURE



- Collect data in real-time
- Store, analyze and visualize
- Set alerts and thresholds
- Act before users are impacted

5. TOOLS YOU CAN USE



Use the right tool based on your stack and needs.

6. COMMON PITFALL



OVER-LOGGING

Logging everything can cause:

- ❌ High storage costs
- ❌ Difficult debugging
- ❌ Noise hides important issues
- ❌ Performance impact

Instead:

- ✅ Log meaningful events
- ✅ Use appropriate log levels (Information, Warning, Error)
- ✅ Add correlation IDs

7. PRODUCTION INSIGHT

Top teams use monitoring & observability to:

- ✅ Detect issues early
- ✅ Identify performance bottlenecks
- ✅ Understand user behavior
- ✅ Reduce MTTR (Mean Time To Recover)
- ✅ Improve reliability and user satisfaction

Example:

A payment API shows high latency.
Tracing reveals the call to Payment Service is slow due to network latency.
Root cause found in minutes, not hours.

8. PRO TIP

Monitor **BUSINESS METRICS** too.
Not just server & technical metrics.

Examples:

- 🛒 Orders Placed
- 💰 Payments Processed
- 👤 Failed Logins
- 👤 Active Users

Business metrics show the real health of your application.

9. INTERVIEW INSIGHT

Q. What is the difference between Monitoring and Observability?

Monitoring

Detects that a problem exists.

Observability

Helps explain why the problem exists.

“Monitoring is knowing that something is wrong.
Observability is figuring out what is wrong and why.”

10. BENEFITS vs WITHOUT MONITORING

WITH MONITORING & OBSERVABILITY

- ✅ Faster issue detection
- ✅ Easier root cause analysis
- ✅ Better system reliability
- ✅ Proactive performance improvements
- ✅ Happy users 😊

VS

WITHOUT MONITORING

- ❌ Longer outages
- ❌ Reactive firefighting
- ❌ Difficult troubleshooting
- ❌ Higher operational cost
- ❌ Poor customer experience 😞

11. KEY TAKEAWAY

- ✅ Monitoring tells you **WHEN** something is wrong
- ✅ Observability helps you understand **WHY**
- ✅ Use Logs, Metrics and Traces together
- ✅ Monitor your business, not just infrastructure
- ✅ A well-observed system is a reliable system

You can't improve what you can't measure.

NEXT UP (DAY 91)

System Design Basics –
Designing layered individual APIs
and designing scalable systems.



SCENARIO FOR YOU

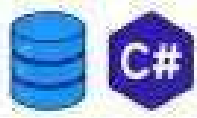
If users report that your API is slow,
would you rather guess the cause
or use logs, metrics, and traces
to identify it quickly?



REMEMBER

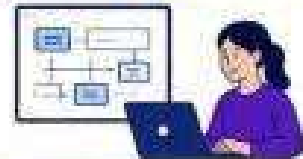
- ✅ Document early
- ✅ Monitor continuously
- ✅ Observe deeply
- ✅ Improve constantly





Thinking beyond individual APIs

Designing systems that are scalable, reliable, and ready for growth.



1 REAL-WORLD SCENARIO

You are building an e-commerce application.

On a normal day:

500 users visit your website.



During a festival sale:

500,000 users arrive within minutes!



Would the same architecture still work? Probably not.

That's where System Design becomes essential.

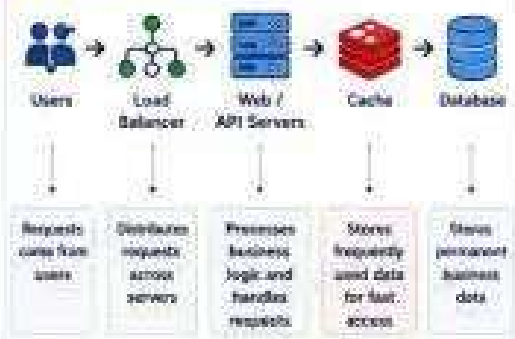
2 WHAT IS SYSTEM DESIGN?

System Design is the process of designing software systems that are:



Instead of thinking about:
• One API
You think about:
• The entire system.

3 HIGH-LEVEL ARCHITECTURE



★ Every layer has a specific responsibility.

4 CORE COMPONENTS

1. LOAD BALANCER

Distributes incoming requests across multiple servers to ensure high availability.

- Examples:
- Azure Load Balancer
 - Application Gateway
 - Nginx, HAProxy

2. APPLICATION SERVERS

Process business logic, handle client requests and return responses.

- Examples:
- ASP.NET Core Web API
 - Multiple instances behind the load balancer

3. CACHE

Stores frequently accessed data to reduce database load and improve response time.

- Examples:
- Redis
 - Distributed Cache
 - In-Memory Cache

4. DATABASE

Stores permanent, transactional data required by the application.

- Examples:
- SQL Server
 - Azure SQL Database
 - Cosmos DB

5 COMMON PITFALL

Scaling only the database.
Many developers focus on:
• Bigger database

Instead focus on:
• Load balancing
• Caching
• Horizontal scaling
• Removing bottlenecks



6 PRODUCTION INSIGHT

Most modern applications don't scale by buying one bigger server. They scale by:

- Adding more application instances
- Using distributed caching
- Decoupling services
- Optimizing bottlenecks

7 PRO TIP

Before optimizing code, identify the bottleneck. Sometimes it is:

- Database
- Network
- External APIs

Not every performance issue comes from your application code.

8 INTERVIEW INSIGHT

Q. What is the primary goal of System Design?
A. Building systems that continue performing well as users, traffic, and data grow.

It's not just about making it work. It's about making it work at scale.

9 GOOD DESIGN vs POOR DESIGN

GOOD DESIGN

- Scalable and elastic
- Fault tolerant
- High availability
- Performs under load
- Easy to maintain
- Loose coupling between services

VS

POOR DESIGN

- Single points of failure
- Difficult or expensive to scale
- Low availability
- Performance bottlenecks
- Hard to maintain
- Tight coupling between components

10 KEY TAKEAWAY

- System Design is about making applications ready for growth, failures, and high traffic — not just making them work.
- Think about the entire system, not just a single API or component.
- Design for scalability, reliability, and maintainability from day one.
- Identify bottlenecks, design for failures, and keep improving.



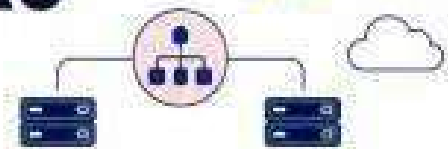
CHALLENGE FOR YOU

If your application suddenly receives 100+ more traffic overnight, which part of your architecture would you improve first, and why?





How modern applications handle millions of requests



1 REAL-WORLD SCENARIO

Your application runs on a single server.

Everything works... until a marketing campaign goes viral!



100,000 users arrive at the same time

- Server becomes overloaded
- Requests start timing out
- Users leave

Companies don't rely on a single server. They use Load Balancers!

2 WHAT IS A LOAD BALANCER?

A Load Balancer sits between users and your application servers.

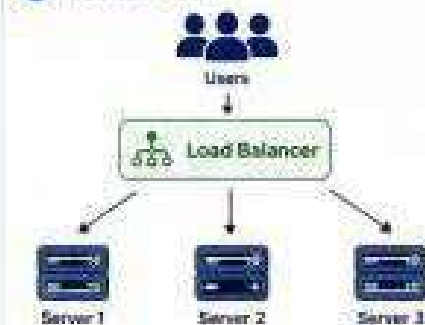
Its job is simple:

- Receive incoming requests
- Distribute them across multiple servers
- Prevent any one server from becoming overloaded



Ensures optimal resource utilization, high availability, and great user experience.

3 REQUEST FLOW



Requests are distributed across multiple servers instead of sending all traffic to one.

4 WHY USE LOAD BALANCERS?



Better Performance



High Availability



Fault Tolerance



Easier Horizontal Scaling



Reduced Downtime

5 COMMON LOAD BALANCING ALGORITHMS

ROUND ROBIN

Each request goes to the next server in sequence.

- User 1 → Server A
- User 2 → Server B
- User 3 → Server C
- User 4 → Server A

Simple and widely used.

LEAST CONNECTIONS

Requests are sent to the server with the fewest active connections.

- Server A: 2
- Server B: 0
- Server C: 1

Useful when requests have different processing times.

IP HASH

The same client is routed to the same server based on its IP address.

- 192.168.1.10 → Server A
- 192.168.1.20 → Server B
- 192.168.1.30 → Server A

Useful for session-based applications.

6 COMMON PITFALL

Assuming a Load Balancer alone makes your application scalable.



It doesn't!

Your application should also be:

- Stateless
- Horizontally scalable
- Able to share session data if required



7 PRODUCTION INSIGHT

Cloud platforms provide managed load balancing services:

- Azure Load Balancer
- Azure Application Gateway
- AWS Elastic Load Balancing (ELB)
- NGINX
- HAProxy



Most enterprise applications rely on one of these instead of building their own.

8 PRO TIP



Design your APIs to be stateless whenever possible.



Stateless services can be moved between servers without affecting users, making scaling much easier.

9 INTERVIEW INSIGHT

Q: What happens if one server behind a Load Balancer fails?

The Load Balancer performs health checks and automatically routes traffic only to healthy servers, improving availability.



10 BENEFITS vs CHALLENGES

BENEFITS

- Better performance
- High availability
- Fault tolerance
- Easier scaling
- Reduced downtime

VS

CHALLENGES

- Additional infrastructure
- Configuration complexity
- Session management considerations

10 KEY TAKEAWAY

- Load Balancers improve performance, reliability, and scalability.
- They distribute traffic across multiple servers.
- They are a fundamental building block of modern cloud-native architectures.



NEXT UP (DAY 93)

Horizontal vs Vertical Scaling — choosing the right strategy as your application grows.



CHALLENGE FOR YOU

If one of your application servers suddenly goes offline, how should the system continue serving users without noticeable downtime?



REMEMBER

- Health Checks
- Distribute Traffic
- Failover Automatically
- Keep Users Happy



HORIZONTAL vs VERTICAL SCALING

Choosing the right way to scale your application



1. REAL-WORLD SCENARIO

Your e-commerce application suddenly receives 10x more traffic during a festival sale.

Users start reporting:

- Slow page loads
- API timeouts
- Failed checkouts

How do you handle the increased load?

2. WHAT IS SCALING?

Scaling means increasing your application's capacity to handle more users, requests, or data.

There are two common approaches:

Vertical Scaling (Scale Up)

Horizontal Scaling (Scale Out)

3A. VERTICAL SCALING (SCALE UP)



4 CPU
16 GB RAM

16 CPU
64 GB RAM

Upgrade your existing server:
One bigger machine.

Advantages

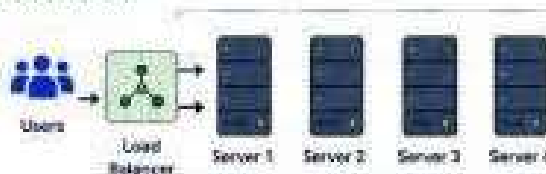
- Simple to implement
- No application changes
- Works well for smaller systems

Limitations

- Hardware limits
- More expensive over time
- Single point of failure

3B. HORIZONTAL SCALING (SCALE OUT)

Instead of one powerful server, add more servers and distribute traffic using a Load Balancer.



Advantages

- High availability
- Better fault tolerance
- Handles massive traffic
- Easier long-term growth

Challenges

- More infrastructure
- Session management
- Distributed caching
- Data consistency

4. VERTICAL vs HORIZONTAL SCALING - COMPARISON

Aspect	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
How it works	Increase power of a single server	Add more servers and distribute load
Example	4 CPU → 16 CPU	Add Server 1, Server 2, Server 3...
Setup	Easier and faster	More complex
Scalability	Limited by hardware capacity	Nearly unlimited
Availability	Higher risk of downtime	Better availability and fault tolerance
Cost (Long term)	More expensive as you scale up	More cost-effective as you scale out
Best for	Small apps, low traffic, quick needs	Large apps, high traffic, cloud-native systems

5. COMMON PITFALL

Scaling the application without identifying the bottleneck.

Sometimes the issue isn't the API. It could be:

- Database
- Network
- Cache
- External services

Always measure before scaling.

6. PRODUCTION INSIGHT

Large platforms like Netflix, Amazon, and Uber primarily rely on horizontal scaling because adding more servers is more practical than endlessly upgrading a single machine.

NETFLIX amazon Uber

7. PRO TIP

Don't optimize prematurely.

START SIMPLE

Scale only when your metrics show it's necessary.



8. INTERVIEW INSIGHT

Q: When would you choose Vertical Scaling over Horizontal Scaling?

Answer:

- Vertical Scaling is often suitable for:
- Small applications
 - Legacy systems
 - Low-traffic workloads
 - Quick infrastructure upgrades

- Horizontal Scaling is preferred for:
- Cloud-native applications
 - High availability
 - Large-scale systems

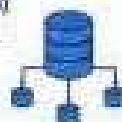
9. KEY TAKEAWAY

- Vertical Scaling = Bigger machine (Scale Up)
- Horizontal Scaling = More machines (Scale Out)
- Choosing the right strategy depends on your application's architecture, traffic patterns, and availability requirements.



NEXT UP (DAY 94)

Database Sharding - Splitting data for massive scale



SCENARIO FOR YOU

If your application suddenly receives 5 million users overnight, would you first upgrade one server or distribute the workload across multiple servers? Why?



REMEMBER

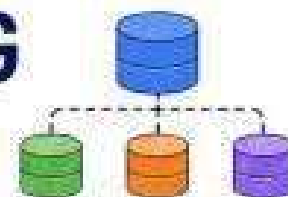
- Measure
- Identify bottlenecks
- Choose the right scaling strategy
- Monitor continuously



DATABASE SHARDING

Splitting data to scale millions of users

Scale your database horizontally, not just vertically.



1. REAL-WORLD SCENARIO

Your application has grown to **50 MILLION USERS**.

The database is experiencing:

- Slow queries
- High CPU usage
- Storage limitations
- Long backup times

Even after optimizing indexes and queries, performance is still degrading.

What next? → Database Sharding!

2. WHAT IS DATABASE SHARDING?

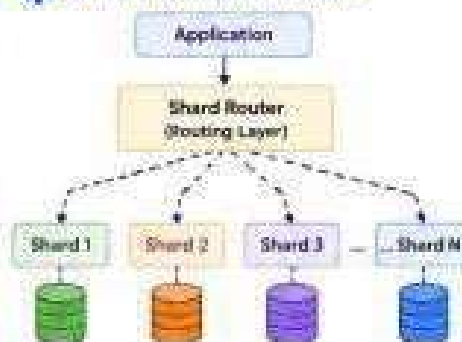
Database Sharding is the process of splitting one large database into multiple smaller databases called shards.

Each shard stores only a portion of the data.

Users 1 – 1M	→	Shard 1
Users 1M – 2M	→	Shard 2
Users 2M – 3M	→	Shard 3
Users 3M+	→	Shard 4

Instead of one overloaded database, the workload is distributed.

3. HOW SHARDING WORKS



The application (or routing layer) decides which shard should receive the request.

4. COMMON SHARDING STRATEGIES

A. RANGE-BASED SHARDING

Data is split based on a range of values.

UserId: 1 – 1M	→	Shard 1
UserId: 1M – 2M	→	Shard 2
UserId: 2M – 3M	→	Shard 3
UserId: 3M+	→	Shard 4

Simple to implement but may create uneven distribution.

B. HASH-BASED SHARDING

Data is distributed using a hash function.



Provides a more balanced distribution across shards.

C. GEOGRAPHIC SHARDING

Data is split based on geographic regions.

India	→	Shard A
Europe	→	Shard B
USA	→	Shard C
Australia	→	Shard D

Useful for global applications; reduces latency and improves compliance.

5. BENEFITS vs CHALLENGES

BENEFITS

- Handles massive datasets
- Better availability
- Faster queries
- Reduced database contention
- Independent scaling

VS

CHALLENGES

- Cross-shard joins are difficult
- Distributed transactions
- Increased operational complexity
- Data rebalancing
- More complex monitoring & backup

6. COMMON PITFALL

Sharding too early.
Many applications don't need sharding.

Optimize first:



Only shard when a single database becomes the bottleneck.

7. PRODUCTION INSIGHT

Large platforms like Instagram, Uber, and Shopify use sharding because a single database cannot efficiently handle their scale.

However, sharding is usually introduced after exhausting simpler optimization techniques.



8. PRO TIP

Sharding is an architectural decision – not just a database feature.

It impacts:

- Application logic
- Deployment
- Monitoring



9. INTERVIEW INSIGHT

Q. What's the difference between Replication and Sharding?

REPLICATION



Copies the same data to multiple databases for availability and read scalability.

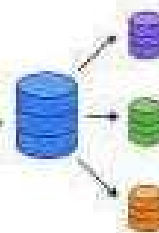
SHARDING



Splits different portions of data across multiple databases for write scalability and storage growth.

10. KEY TAKEAWAY

- Replication improves availability.
- Sharding improves scalability.
- Scaling isn't always about making one database bigger.
- Sometimes it's about splitting the workload intelligently.



NEXT UP (DAY 95)

CAP Theorem – Why distributed systems can't have everything



SCENARIO FOR YOU

If your database server reaches its maximum capacity, would you upgrade it again, add read replicas, or split the data into shards? What factors would influence your decision?



REMEMBER

- Measure before scaling.
- Understand your bottleneck.
- Choose the right strategy.
- Design for growth.





CAP THEOREM EXPLAINED

Why distributed systems can't have everything



1. REAL-WORLD SCENARIO

Your application runs across multiple servers in different regions.

Suddenly...



A network failure occurs between two data centers.

Now your system has to choose:



Keep accepting requests?



Keep all data perfectly consistent?



You can't always have both. That's where the CAP Theorem comes in.



2. WHAT IS THE CAP THEOREM?

The CAP Theorem states that a distributed system can guarantee only TWO of these THREE properties during a network partition.



CONSISTENCY

Every user sees the same latest data.



AVAILABILITY

Every request receives a response, even if some data isn't the latest.



PARTITION TOLERANCE

The system continues working even if communication between servers is interrupted.

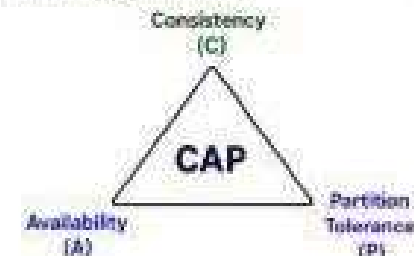


In a partition, you can have only TWO, not all THREE.



3. THE THREE CHOICES

During a network partition, you must choose one of these combinations:



You can have:

CA + P or A + P

But **NEVER** all three together.



4. CP vs AP COMPARISON

	CP (Consistency + Partition Tolerance)	AP (Availability + Partition Tolerance)
You Choose	Consistency Partition Tolerance	Availability Partition Tolerance
Result During Partition	Some requests may be rejected until data is synchronized.	All requests are accepted, but some users may see stale (old) data.
Example	Banking systems (Transaction accuracy is critical)	Social media feeds (User experience is critical)
Analogy	"Better to be safe than sorry."	"Better to be fast than perfect."



5. REAL-WORLD EXAMPLES

Consistency First (CP)



Banking / Finance Systems

- Accurate balances
- No incorrect transactions
- May reject requests during failures

Availability First (AP)



Social Media / Feeds Chat / Notifications

- Always available
- Some data may be outdated
- Better user experience

E-commerce? It depends on the operation!
Catalog → Availability | Payments → Consistency



6. COMMON PITFALL

Thinking CAP means you permanently choose only two.



Not exactly!
The trade-off matters **ONLY** when a network partition occurs.

Under normal conditions, systems often provide all three reasonably well.



7. PRODUCTION INSIGHT

Different systems prioritize different goals:



Banking → Consistency First



Social Media → Availability First



E-commerce → Depends on the operation



Great systems make the right trade-off based on the business need.



8. PRO TIP

The right choice depends on your business requirements — not the technology.



Ask yourself:
What is more harmful?

- Returning stale (old) data?
- Or rejecting requests?



9. INTERVIEW INSIGHT

Q. Why is Partition Tolerance usually considered mandatory?

Because in distributed systems, network failures are inevitable.

The real design decision is: Consistency vs Availability when partitions happen.



10. BENEFITS vs TRADE-OFFS

Prioritize Consistency (CP)

- ✓ Accurate data
- ✓ Reliable transactions
- ✓ Strong correctness
- ✓ Trust & Integrity



Trade-off:
Reduced availability during failures

VS

Prioritize Availability (AP)

- ✓ Better uptime
- ✓ Faster user experience
- ✓ Always responsive
- ✓ High availability



Trade-off:
Temporary data inconsistency



11. KEY TAKEAWAY



During a network partition, a distributed system must balance Consistency and Availability while maintaining Partition Tolerance.

CAP Theorem isn't about finding a perfect system — it's about understanding the trade-offs every distributed system must make.



NEXT UP (DAY 96)

Consistency vs Availability — Understanding real-world distributed systems



SCENARIO FOR YOU

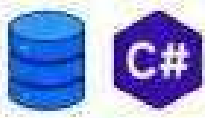
If you're designing an online banking system, would you prefer temporarily rejecting transactions or allowing users to see inconsistent account balances?



REMEMBER

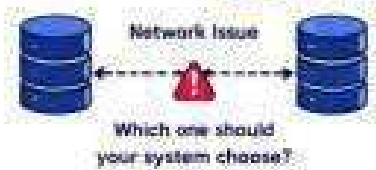
- Understand the trade-offs
- Choose based on business needs
- Design for failure, always





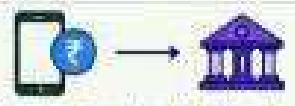
CONSISTENCY VS AVAILABILITY

Making the right trade-off in distributed systems



1. REAL-WORLD SCENARIO

In an online banking application, a network issue occurs between two database servers.



A customer transfers ₹10,000 and immediately checks the balance.

Should the system:

Wait until all servers agree on the latest balance? (Consistency)

OR

Return the last known balance immediately? (Availability)

2. WHAT IS CONSISTENCY?

Consistency means every user reads the same, most recent data.

Example:

Customer transfers ₹10,000.

Every server returns the updated balance.

No stale data.



3. WHAT IS AVAILABILITY?

Availability means every request receives a response.

Even if some servers haven't synchronized yet.

Example:

A user may briefly see the previous balance, but the application remains responsive.



4. CONSISTENCY vs AVAILABILITY

CONSISTENCY	VS	AVAILABILITY
Every user sees the latest data	⌚ Focus	Every request receives a response
May reject requests during failures	🛡️ During failure	Always responds, even with stale data
High data accuracy	🎯 Goal	High system uptime & responsiveness
Ideal for financial systems	★ Best for	Ideal for social platforms

5. REAL-WORLD EXAMPLES



BANKING SYSTEMS

Priority: **CONSISTENCY**

Incorrect balances are unacceptable.

- All transactions must be accurate
- Strong consistency is required
- May reject requests during failures

Example: Fund transfer, account balance, loan processing



SOCIAL MEDIA

Priority: **AVAILABILITY**

If a post appears a few seconds later, user experience is still acceptable.

- Always available
- Small delays are okay
- May return slightly outdated data

Example: Feeds, likes, comments, notifications



E-COMMERCE

Priority: **DEPENDS ON OPERATION**

Different operations choose differently.

- Product catalog → Availability
- Inventory updates → Availability
- Payment processing → Consistency
- Order placement → Consistency

Example: Search & browse vs payments & order creation

6. COMMON PITFALL

Assuming every part of an application needs the same consistency level.



Different features often have different requirements.

One size does NOT fit all in distributed systems.

7. PRODUCTION INSIGHT

Modern distributed systems choose based on the business operation.



Business Operation



Choose the Right Trade-off



Better User Experience



Large platforms stay fast while still protecting critical transactions.

8. PRO TIP

Ask one question:



What is more expensive for the business?

- Returning slightly outdated data?
- Or rejecting user requests?

Your answer determines the right architecture.

9. INTERVIEW INSIGHT

Q: Does CAP Theorem mean you should always choose Availability?

No.

The correct choice depends on:

- Business requirements
- User expectations
- Data criticality



10. TRADE-OFFS

PRIORITIZE CONSISTENCY (CP)

- Accurate data
- Reliable transactions
- Strong correctness

- Lower availability during failures

Typical Use Cases

- Banking systems
- Payment gateways
- Audit & compliance
- Financial reporting

VS

PRIORITIZE AVAILABILITY (AP)

- Better user experience
- Higher system uptime
- Faster responses

- Temporary stale data (eventual consistency)



Typical Use Cases

- Social media
- Chat applications
- Content delivery
- E-commerce catalog



11. KEY TAKEAWAY

- Consistency ensures users always see the latest data.
- Availability ensures users always receive a response.
- Great system design is about choosing the right balance for each business scenario.



NEXT UP (DAY 97)

Top Backend Interview Mistakes (and how to avoid them)



SCENARIO FOR YOU

If you're designing a payment gateway, would you rather reject a transaction temporarily or risk sharing an incorrect account balance?



REMEMBER

- Understand the trade-offs
- Choose based on business needs
- Design for future, always



KEEP LEARNING!

Every trade-off you understand today makes you a better engineer tomorrow.

#SrinjaBuilds



LESSONS

I WISH I KNEW EARLIER



• 97 Days of Learning • Real Experience • Practical Lessons •

1



Write code for the next developer.

That developer might be you six months later.

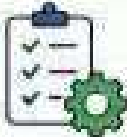
6



Every abstraction has a cost.

Don't use patterns because they're popular. Use them because they solve a real problem.

2



Make it work. Then make it clean. Then make it fast.

Don't optimize code that doesn't work.

7



Readability beats cleverness.

Simple code is easier to review, test, and maintain.

3



Understand the database.

Many "slow APIs" are actually slow SQL queries.

8



Security isn't optional.

- Validate input.
- Protect secrets.
- Follow least privilege.
- Never trust user input.

4



Don't memorize frameworks. Understand the concepts behind them.

Frameworks evolve. Fundamentals stay.

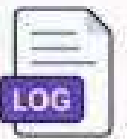
9



Learn from feedback.

Some of the best improvements in this series came from experienced engineers who shared their perspectives in the comments. Constructive feedback makes us better developers.

5



Logging is part of the feature.

If you can't debug production, the feature isn't complete.

10



Keep learning in public.

You don't need to know everything before sharing. Learning consistently and helping others creates opportunities you never expected.



PRODUCTION INSIGHT



Software engineering isn't about writing more code.

It's about solving problems with reliable, maintainable, and scalable solutions.



PRO TIP

Invest in fundamentals:

- Databases
- Networking
- System Design
- Clean Code

Frameworks become much easier afterwards.



KEY INSIGHT



The best engineers aren't the ones who know the most.

They're the ones who never stop learning.



KEY TAKEAWAY



These lessons have shaped how I approach backend development, and I hope they help someone else on their journey too. ❤️



NEXT UP (DAY 98)

How Learning in Public Changed My Career



SCENARIO FOR YOU

Which backend engineering lesson has had the biggest impact on your career?



REMEMBER

- Keep Learning
- Keep Building
- Keep Sharing
- Keep Growing



DAY 98

10 LESSONS

THAT MADE ME A BETTER BACKEND ENGINEER

N.NET



— 98 days of learning, building, sharing & growing



1



CLEAN CODE IS WRITTEN FOR PEOPLE, NOT COMPILERS.

Readable code is easier
to maintain than clever code.

6



LOGGING SAVES HOURS OF DEBUGGING.

Future you will thank
present you.

2



PERFORMANCE STARTS WITH MEASUREMENT.

Never optimize based
on assumptions.
Measure first.

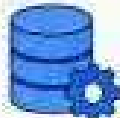
7



SIMPLE SOLUTIONS OFTEN SCALE BETTER THAN COMPLEX ONES.

Complexity should solve real
problems, not create new ones.

3



UNDERSTANDING DATABASES IS AS IMPORTANT AS WRITING APIS.

A fast API can't hide
a slow database.

8



GREAT APIS ARE PREDICTABLE.

Consistency is one of the
best developer experiences
you can provide.

4



SECURITY ISN'T A FEATURE.

It should be part of every
design decision.

9



LEARNING IN PUBLIC ACCELERATES GROWTH.

Teaching a concept forces
you to truly understand it.

5



EVERY ABSTRACTION HAS A COST.

Use design patterns when
they solve problems—
not because they're popular.

10



THE BIGGEST LESSON...

There will always be more to learn.
Technology evolves.
The best engineers stay curious.



THE BIGGEST LESSON

STAY CURIOUS.

Keep learning. Keep building. Keep growing.
That's the real journey.



MY JOURNEY SNAPSHOT (98 DAYS)



C#



N.NET



ASP.NET Core



SQL Server



EF Core



API Design



Security



Architecture



Over these 98 days, I've shared topics covering backend fundamentals to advanced concepts.
Every discussion, correction, and comment from this community has helped me grow.
Thank you for being part of this journey. ❤️



I'D LOVE TO HEAR FROM YOU!

What's one lesson that made
you a better software engineer?

👉 Share it in the comments.



The best engineers
are lifelong learners.



ONE STEP CLOSER TO DAY 100! 🚀

Let's finish strong
together!

#dotnet

#backenddevelopment

#aspnetcore

#100DaysOfCode

#LearningInPublic

#SrinijaBuilds

DAY 99

ONE DAY TO GO!

99 Days of LEARNING IN PUBLIC

A JOURNEY OF CONSISTENCY, GROWTH & GRATITUDE

**“Consistency
beats motivation.”**



DAY 1

The start of
a commitment
to grow every day



99

TECHNICAL POSTS

Shared learnings
consistently across
99 days.



1,000+

LINKEDIN FOLLOWERS

Grateful for the support,
encouragement and
trust you all showed.



VALUABLE
COMMUNITY FEEDBACK

Every comment, suggestion
and correction helped me
learn better.



CONTINUOUS
LEARNING

Explored, practiced and
grew a little better
every single day.



THINGS I LEARNED BEYOND CODING

- Accept feedback with an open mind.
- It's okay to simplify concepts—as long as you keep learning.
- The best discussions often happen in the comments.
- Teaching a concept is one of the fastest ways to understand it better yourself.



Small steps
every day
lead to big
transformation.



TOPICS I EXPLORED

- C# & .NET Fundamentals
- ASP.NET Core & Web APIs
- Entity Framework Core
- SQL Server
- API Design & Best Practices
- Security & Authentication
- Architecture & Design Patterns
- Azure & Cloud Concepts
- ...and so much more!



MILESTONES I'M GRATEFUL FOR



Connected with
incredible developers
across the globe



Improved my technical
knowledge and
communication skills



Built confidence through
consistency and
community



DAY 99

Still learning.
Still growing.
Still grateful.

THANK YOU!

To everyone who liked, commented, shared, encouraged and corrected me—
thank you for being part of this beautiful journey.

This journey is ours.

Together, we grow!



ONE MORE DAY UNTIL THE FINISH LINE.

Excited for Day 100!



#SrinijaBuilds





Beyond 100 Days

The biggest achievement wasn't completing 100 topics. It was developing the habit of learning every single day. These 100 days reinforced that...

- 💡 Consistency beats motivation.
- 💡 Small progress every day compounds into big results.
- 💡 Learning in public accelerates growth.
- 💡 Feedback helps you become a better engineer.
- 💡 Understanding concepts is more valuable than memorizing answers.
- 💡 Senior engineers connect concepts—they don't just know definitions.
- 💡 Production experience starts with strong fundamentals.
- 💡 There is always something new to learn.
- 💡 Great software is built through continuous improvement.
- 💡 Knowledge grows when it's shared.

If this guide helped you...

★ Share it with your friends and colleagues preparing for .NET or Backend interviews.



Repost it so more developers can benefit.



Follow me for more interview guides, backend engineering content, and practical learning resources.

Let's continue learning, building, and growing together.

Acknowledgements

This guide is a personal compilation and organization of the excellent 100 Days of Backend Engineering series created by Srinija Velaga.

Special thanks for the time, consistency, and effort invested in creating valuable content for the developer community.

Compiled & Organized by Shama Shaik

"Keep Learning. Keep Building. Keep Sharing."